Soyeon Choi

F26-008-9_Part_2_Chapters_6-10_and_Appendix_with_cover (1)-1-

University of Science and Technology Graduate School

Document Details

Submission ID
trn:oid:::3971:158451361

Submission Date
Nov 13, 2026, 4:27 PM GMT+9

Download Date
Nov 13, 2026, 4:29 PM GMT+9

File Name
F26-008-9_Part_2_Chapters_6-10_and_Appendix_with_cover (1)-1-1.docx

File Size
2.6 MB

72 Pages

19,404 Words

99,483 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

70 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
1% Publications
1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

70 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
1% Publications
1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Universitat Politècnica de València on 2016-11-11 — 1%
2	Internet	assignmentessays.com — <1%
3	Internet	ijisrt.com — <1%
4	Internet	ualberta.scholaris.ca — <1%
5	Submitted works	Universitat Ramon Llull on 2022-09-09 — <1%
6	Submitted works	Universitat Pompeu Fabra on 2014-03-04 — <1%
7	Internet	gupea.ub.gu.se — <1%
8	Submitted works	Universitat Politècnica de València on 2012-11-12 — <1%
9	Internet	e-archivo.uc3m.es — <1%
10	Internet	orca.cardiff.ac.uk — <1%

11 Submitted works
Universitat Pompeu Fabra on 2019-04-17 <1%

12 Submitted works
Universitat Politècnica de València on 2023-06-22 <1%

13 Submitted works
Universitat Politècnica de València on 2011-06-21 <1%

14 Internet
rua.ua.es <1%

15 Internet
riunet.upv.es <1%

16 Internet
api.mountainscholar.org <1%

17 Submitted works
Universitat Pompeu Fabra on 2017-08-13 <1%

18 Internet
ruc.udc.es <1%

19 Submitted works
Universitat Pompeu Fabra on 2023-12-13 <1%

20 Submitted works
Universitat Politècnica de Catalunya on 2026-06-02 <1%

21 Internet
www.mobt3ath.com <1%

22 Internet
ruc.udc.es <1%

23 Internet
ar5iv.labs.arxiv.org <1%

24 Internet
gupea.ub.gu.se <1%

25 Publication
Arthur Scalercio, Aline Paes. "Masked transformer through knowledge distillation for... <1%

26 Publication
Zhiguang Yang, Gejian Zhao, Hanzhou Wu. "Watermarking for Large Language Model... <1%

27 Internet
curis.ku.dk <1%

28 Internet
gupea.ub.gu.se <1%

29 Internet
etheses.whiterose.ac.uk <1%

30 Internet
epage.pub <1%

31 Internet
gupea.ub.gu.se <1%

32 Internet
gupea.ub.gu.se <1%

33 Publication
Andrea Bruera, Massimo Poesio. "Exploring the Representations of Individual Entities... <1%

34 Submitted works
Universitat Pompeu Fabra on 2021-05-17 <1%

35 Submitted works
Universitat Politècnica de Catalunya on 2022-03-09 <1%

36 Internet
www.econstor.eu <1%

37 Submitted works
University of Hawaii at Manoa on 2021-08-05 <1%

38 Internet
ruc.udc.es <1%

39 Internet

rodin.uca.es <1%

40 Internet

ar5iv.labs.arxiv.org <1%

41 Internet

riunet.upv.es <1%

42 Internet

gupea.ub.gu.se <1%

43 Publication

Elham Seifossadat, Hossein Sameti. "Data-to-text generation using conditional gener... <1%

44 Publication

Shivani Kumar, Sumit Bhatia, Milan Aggarwal, Tanmoy Chakraborty. "Dialogue agent... <1%

45 Submitted works

Universitat Pompeu Fabra on 2018-10-18 <1%





National University of Computer and Emerging Sciences, Lahore



A.R.T.E.M.I.S: Adaptive Reasoning, Task Execution and Multi-agent Intelligence System

Hashir Rauf Roll No. 23L-2572 BS(DS)

Hira Khalid Roll No. 23L-2594 BS(DS)

Syeda Doureesha Batool Roll No. 23L-2651 BS(DS)

Supervisor: Mr. Mubasher Hussain

Final Year Project

November 9, 2026

turnitin Page 8 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Chapter 6 High-Level and Low-Level Design

This chapter presents the design of the system. It begins with an overview and the considerations that constrain the design, then presents the architecture at the layer and component level, the strategies that produced it, the domain model, and the policies and tactics that govern implementation. Diagrams are provided throughout, and each significant decision is traced to the requirement or finding that produced it.

6.1 System Overview

The system is a long-running local process that mediates between a user and a bounded region of their filesystem. It accepts a request by typing or by speech, retrieves context from within the granted workspace, produces a multi-step plan, classifies each step by risk, obtains the user’s approval where required, records how to reverse each change, executes the approved steps, and writes an immutable record of everything proposed and performed.

The design is organised as six layers on the execution path with two planes crossing all of them, as shown in Figure 6.1. The presentation layer provides the command line, graphical and optional remote surfaces over one shared view model. The interaction layer normalises voice and text into a single modalityneutral object. The gateway layer routes model requests, assembles context, classifies risk, plans, and dispatches. The capability layer implements the features. The data layer persists everything locally. The remote access layer is drawn outside the stack because nothing in the layers beneath it depends on its presence.

Layers one to five form the local execution path, the two planes cross every layer, and the remote access plane is drawn outside the stack because removing it leaves a fully functional system.

Security and performance are represented as planes rather than layers, and this is a deliberate representational choice. A layer is a stage a request passes through, and drawing security as a layer would imply that a request could be past security, which is precisely the reasoning error that produces confuseddeputy defects. They are drawn as planes to indicate that every arrow in the architecture crosses both.

turnitin Page 8 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

6.2 Design Considerations

6.2.1 Assumptions and Dependencies

The design depends on the following, each of which would require reconsideration were it to change.

- The canonicalisation procedure of the operating system solves symbolic links in the right way, as the confinement guarantee is constructed on the basis of it. This is all the platform-specific behaviour on which the design relies.

- The relational store maintains referential integrity where it is asked to, as revocation is defined as a cascading deletion. Since the engine switches this off by default, it would need to be turned on in each connection.

- There is a local model runtime or the user has set up cloud fallback on the workspaces in question. Providers are considered configuration by the router and thus provider change is not design change.

- The user is a single person who has control over the folders he/she grants. The design does not deal with shared or simultaneous access by more than one user.

- A document in a workspace can be adversarial, as a file can be created in any place.

- The design thus views all the content retrieved as data and not instruction.

- The capability layer is likely to see the biggest change in functionality since they are added. The reason why the uniform tool contract exists is that the change does not spread to the gateway.

- 6.2.2 General Constraints

- The design was influenced by the following constraints and the effect of each is mentioned.

- Resource envelope. The system should be usable on a consumer laptop, which entails the expensive work is asynchronous and all the queues are limited. The execution plane in Section 6.4 is the result of impact.

- Offline operation. Connectivity should never be part of core functionality, which demands that a local model should be the default tier, and that networked capabilities degrade explicitly. The impact is the degradation contract in Table 6.3

- Data residency. The data of the users should not be sent to a server operated by the project, so the egress is enforced in the router, where workspace policy is known, but not in the tool layer.

- No inbound port. The remote aircraft needs to make an outward dialing hence a topology in Section.

- 6.4 and eliminates the need to configure firewalls or cross network address translation.

- Credential handling. The secrets should be stored in the operating system keyring, which is not part of the system storage tree itself and not backing up of the system.

- Verification. All the guarantees should be testable and this demands that they have a single point of enforcement. This is why there is resolution in one aspect and execution in the other.

- Team and timeline. The architecturally detachable plane is necessitated by the need to have the optional components separable by three members in two semesters.

6.2.3 Goals and Guidelines

The design is dominated by four principles and the purpose behind them is mentioned since none of them is self-explanatory.

12 A house should be built and not advertised. Where it is possible to express a guarantee in terms of the non-existence of a code path, it is expressed in that form. This is the reason why devastating operations are not discussed as to their approval: a defect can defeat a check or a defective route or an unexpected turn, but an incapacity cannot.

There are only one enforcement points per guarantee. Resolution is done in the broker and execution in the dispatcher and not in any other place. This is because of verification: a two-point guarantee is one that has two chances to move apart, and no specific point where it can be verified.

Legibility is a component of trust. Background work reports should be visible, cancellable, the disclosure panel is a live query, and not a written summary, and refusals are recorded in the audit log,

in addition to actions. This is because a system the behaviour of which cannot be observed cannot be trusted no matter how correct it turns out to be.

The refusal failure mode occurs. An operation that has not been recognised is denied, an unsolvable path is rejected, a queue that has reached its full length throws off work into view and an expired approval will never execute. This is because any failure that is not prohibited by a system containing a boundary transforms a defect into a breach.

6.2.4 Development Methods

Design was generated by generating structure based on research results and specification as opposed to modifying a reference architecture. The layered structure with cross-cutting planes is traditional, and the decomposition of the components inside the gateway is due to a particular need: the initial design left routing, context, memory, planning, policy and dispatch to the same component, and this was not testable since no single guarantee could be isolated. The four component decomposition with one common execution point was taken to ensure that each guarantee becomes verifiable. This revision also added the policy engine, as a first-class component, transforming the approval-first finding into enforced structure..

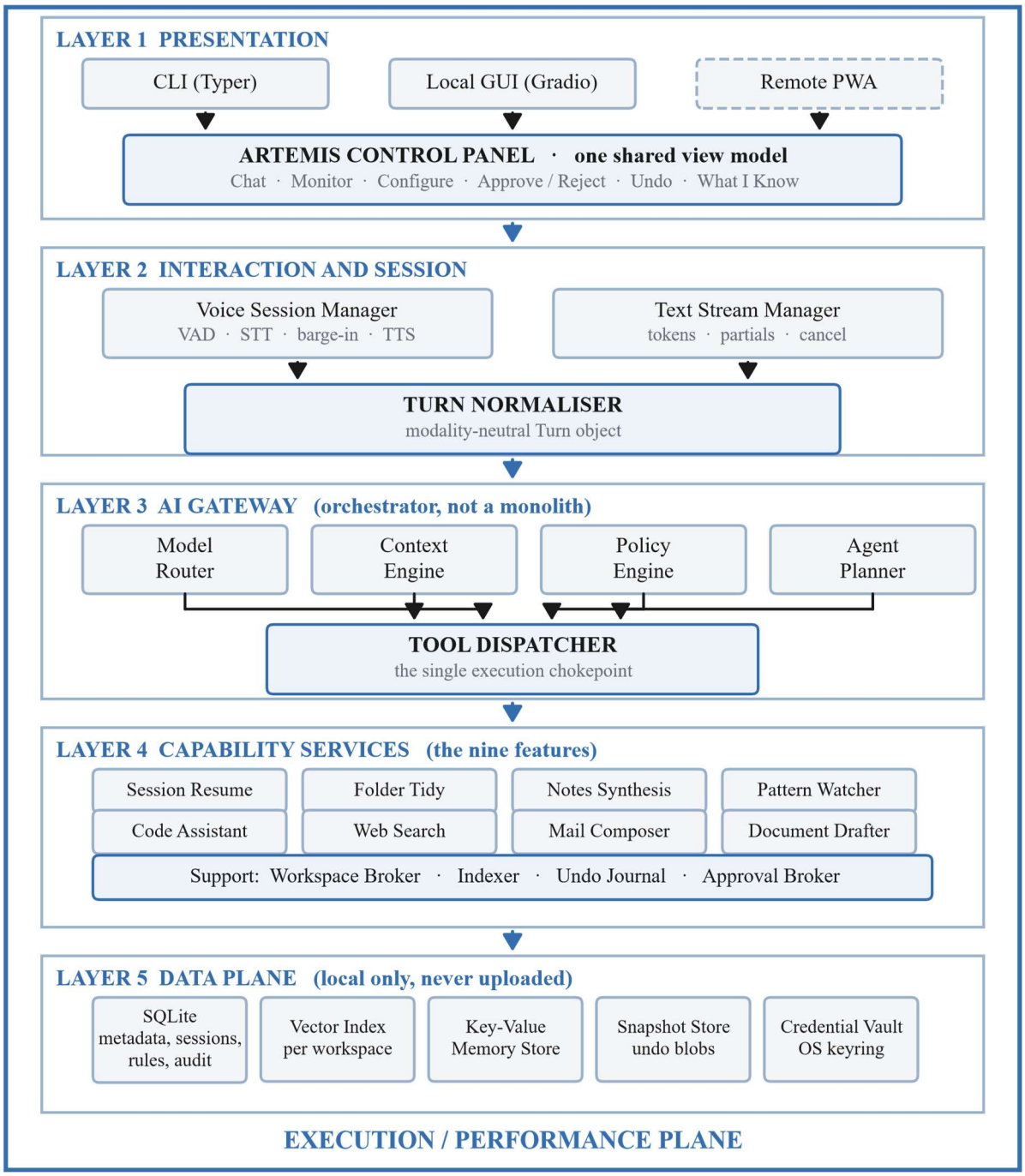


Figure 6.1: The layered architecture.

6.3 System Architecture

The system's responsibilities are partitioned so that each guarantee is held by exactly one component.

Table 6.1 presents the component inventory, stating for each component its layer, its responsibility and

turnitin Page 13 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

the single thing it must never do. The final column is the operative one: it states the invariant whose violation would constitute a defect in the architecture rather than in the implementation.

Table 6.1: Component inventory.

#	Component	Layer	Responsibility	Must never
1	Command line interface	1	Scriptable control surface	Bypass the approval broker
2	Graphical control panel	1	Visual control surface, loopback-bound	Bind to a non-loopback interface
3	Control panel view model	1	Single source of interface truth across surfaces	Hold state the backend does not own
4	Voice session manager	2	Activity detection, transcription, interruption, synthesis	Record when not explicitly armed
5	Text stream manager	2	Token streaming and cancellation	Buffer past a cancellation
6	Turn normaliser	2	Produce the modality-neutral turn	Leak modality downstream
7	Model router	3	Select a tier and fail over; enforce egress	Send workspace data to a disabled tier
8	Context engine	3	Assemble bounded, cited context	Include content outside the grant
9	Policy engine	3	Classify risk and decide gating	Be consulted after execution
10	Agent planner	3	Produce a multi-step plan	Execute anything itself
11	Tool dispatcher	3	Sole execution point	Accept a call lacking a policy verdict
12	Capability services	4	The features	Touch the filesystem directly
13	Workspace broker	4	Path resolution and confinement	Resolve a path outside a grant
14	Indexer	4	Incremental indexing and embedding	Block the interactive path
15	Undo journal	4	Snapshots and inverse operations	Record an action after the fact

Table 6.1: continued from previous page

turnitin Page 13 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

#	Component	Layer	Responsibility	Must never
16	Approval broker	4	Hold pending actions and fan out to surfaces	Approve on timeout
17	Data plane stores	5	Local persistence	Serialise outside the host
18	Remote access plane	6	Authenticated relay	Store or read plaintext user data

19 The collaboration between these components is expressed by the request lifecycle shown in Figure 6.2, which is the single most important path in the system. Every request, whether spoken or typed and whether local or remote, traverses exactly this sequence, and no shortcut exists.

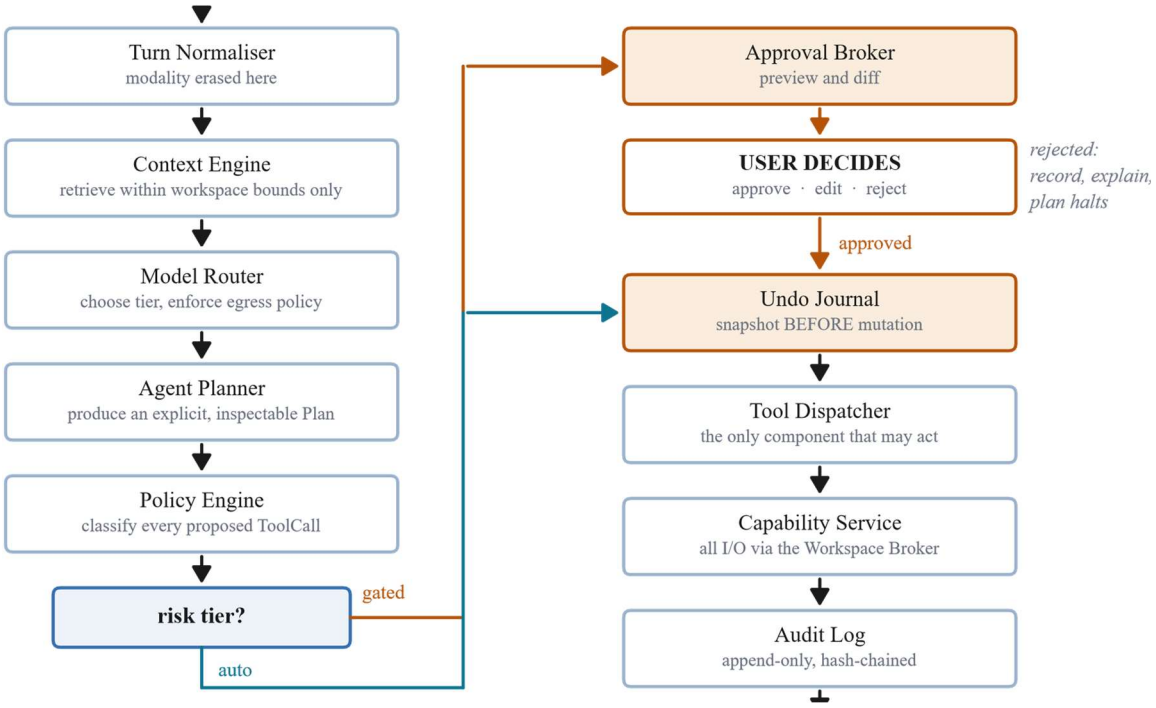


Figure 6.2: The request lifecycle.

The turn normaliser erases modality, policy engine is consulted by the dispatcher, the snapshot comes before the mutation, and audit entry is written regardless of whether action is taken or rejected.

This path has five invariants that are imposed and collectively they make up the guarantees of the architecture. The dispatcher consults the policy engine but never after. The undo journal captures the snapshot of the undo journal prior to the mutation, but never afterwards. Even in the case of action being rejected, the audit entry will be written and therefore the log will capture what was proposed and not just what happened. A call without a corresponding policy

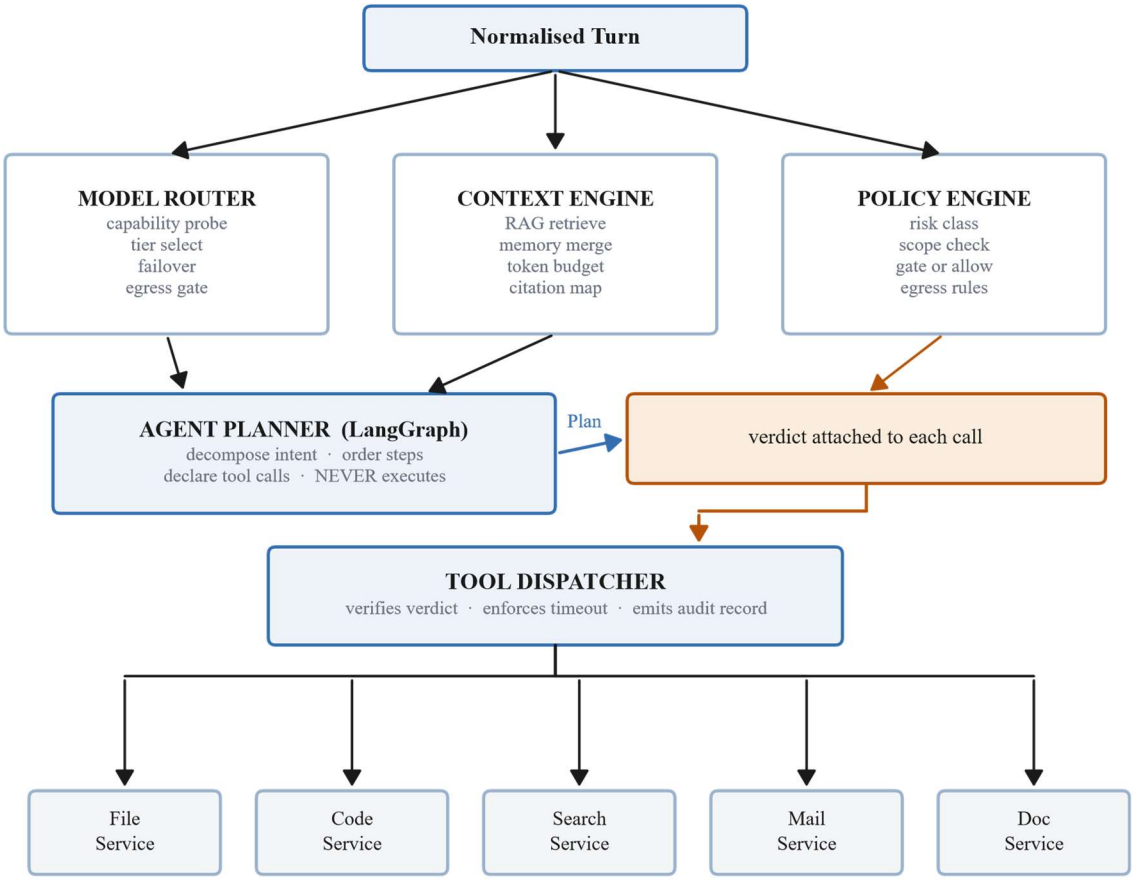


Figure 6.3: Internal structure of the gateway.

verdict is refused by the dispatcher. Voice and text turns are no different than the planner on and that is what makes voice parity structural and not a second implementation.

The decomposition was chosen over two alternatives. One orchestrating component was abandoned due to the impossibility of isolating any single guarantee to be tested, and the fact that a flaw in one component would invalidate all the guarantees at the same time. An arrangement, where each capability had its own gating logic, was not accepted as it would create as many points of enforcement as features, none of which could be centrally audited, and none of which could drift.

6.3.1.1 Subsystem Architecture: The Gateway.

In Figure 6.3, the gateway is broken into four parts having one common point of execution. The planner generates a plan and cannot implement it; the dispatcher implements and cannot generate plans; the policy engine decides on a per-proposed-call basis whether a particular model tier can be used and whether data can leave the machine; and the router decides whether a particular model tier can be used and whether data can leave the machine.

The planner comes up with a plan but never implements it and all the calls end up at the dispatcher, the only point of execution..

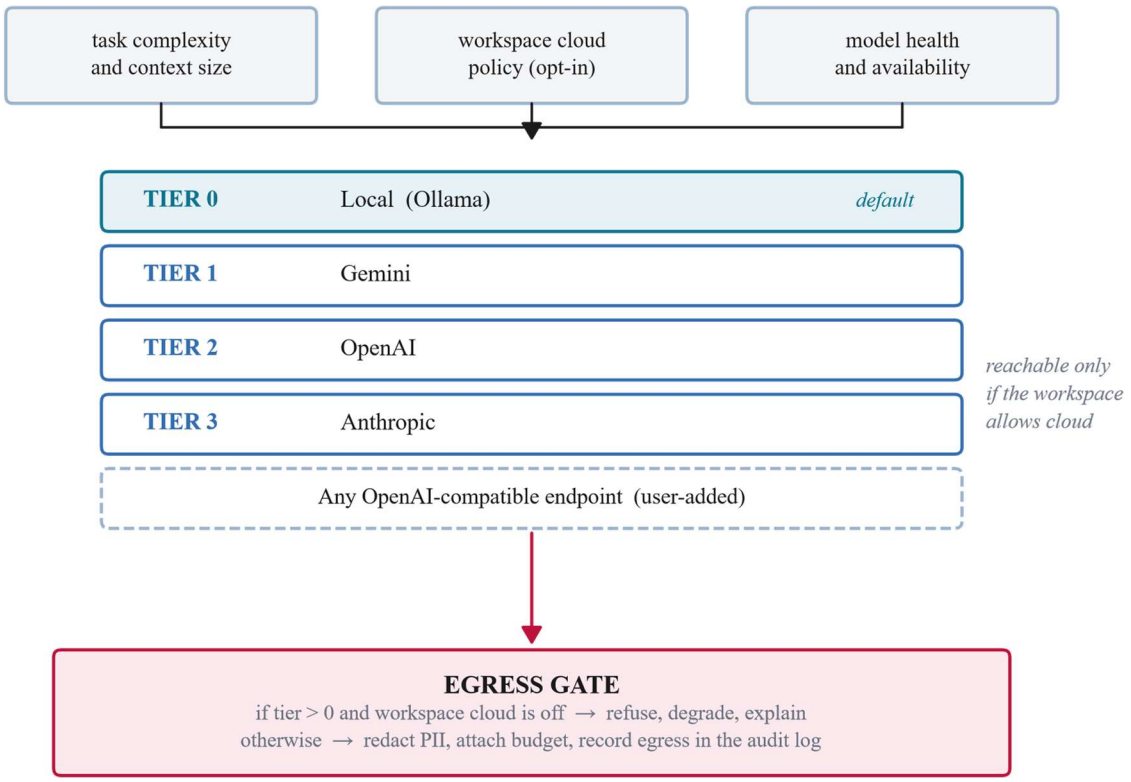


Figure 6.4: Tier selection and the egress gate.

The model router is the owner of tier selection and egress enforcement. The reason why egress is applied here and not in the tool layer or the prompt is that the router is aware of the cloud policy of a particular workspace. The default is the tier zero which is the local model. Later levels can only be accessed where the workspace allows, and any readable endpoint can be added as a new level without any alteration to the code as the providers are considered as configuration, but not as branches. The max retries per tier are defined as one, and only after that do you continue to the next tier, meaning that a workspace that is cloud disabled has only one tier and its failure yields a clean degradation message instead of an undefinite wait. The selection and the egress gate are indicated in figure 6.4.

Higher levels are only accessible in the cases where the cloud policy of the workspace allows and all egress is logged.

The context engine assembles the working context of the model, and imposes limited scope on the read side, just as the broker does on the write side. Each retrieved chunk is identified by the identifier of the workspace where it was retrieved and any chunk which does not match the active grant is discarded and the discard is recorded. The token budgeter is recency-relevant and never truncates a

citation source in silence. Each chunk is stored back to the file and line, in the form of a citation map, and it is this map that makes the synthesis feature verifiable, as a claim that can not be traced by the map is reported as unsupported and not as established. This assembly is depicted in figure 6.6..

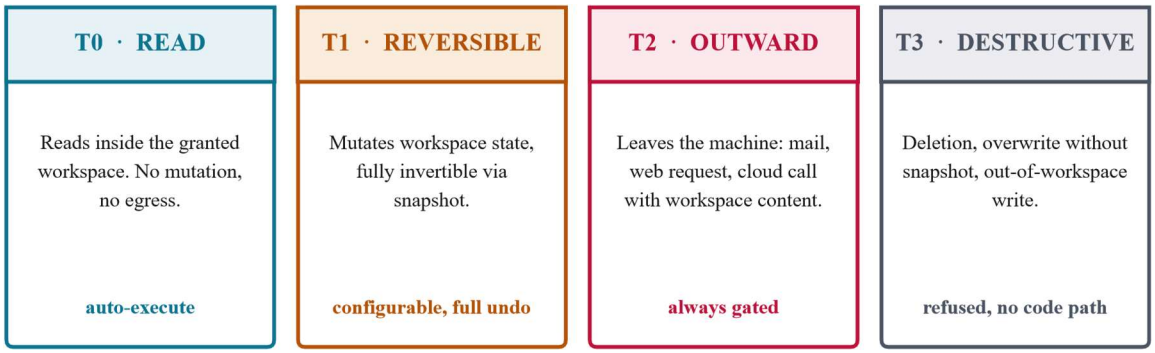


Figure 6.5: The four risk tiers.

The retrieved chunks all bear a workspace identifier, any chunk not within the active grant is deleted and the discard recorded, and the citation map is taken with to the answer.

The policy engine assigns every proposed call in a procedure as declared in Chapter 5. The four levels and the order of evaluation are shown in Figure 6.5 and 6.7 respectively, with denials and gates being earlier than permissions and gates respectively.

The user can only configure the reversible tier, outward actions are always gated and destructive actions are structurally inaccessible.

Denials are considered preceding gates and gates preceding permissions to ensure that the first match is the decision and no subsequent rule can grant what a preceding rule denied.

Figure 6.8 displays a worked example of a folder tidy, where the reading steps are automatically executed and the two state-changing steps are gated, resulting in a preview indicating what will happen, and, no less importantly, what will not.

The operations of reading are automatic and the reversible operations are gated, and the preview states that nothing is taken out of the machine and nothing is lost.

The agent planner generates plans based on LangGraph and never performs them. It has an explicit cycle cap and a wall-clock budget that deals with the repetitive-loop failure mode that the planning literature documents: the inability of an agent to converge presents its partial plan and requests (instead of looping forever).

6.3.2 Subsystem Architecture: Capability Services.

The features are implemented as services behind a uniform tool contract. None accesses the filesystem, the network or the database itself, but rather accesses them via the common ground of the broker, the.

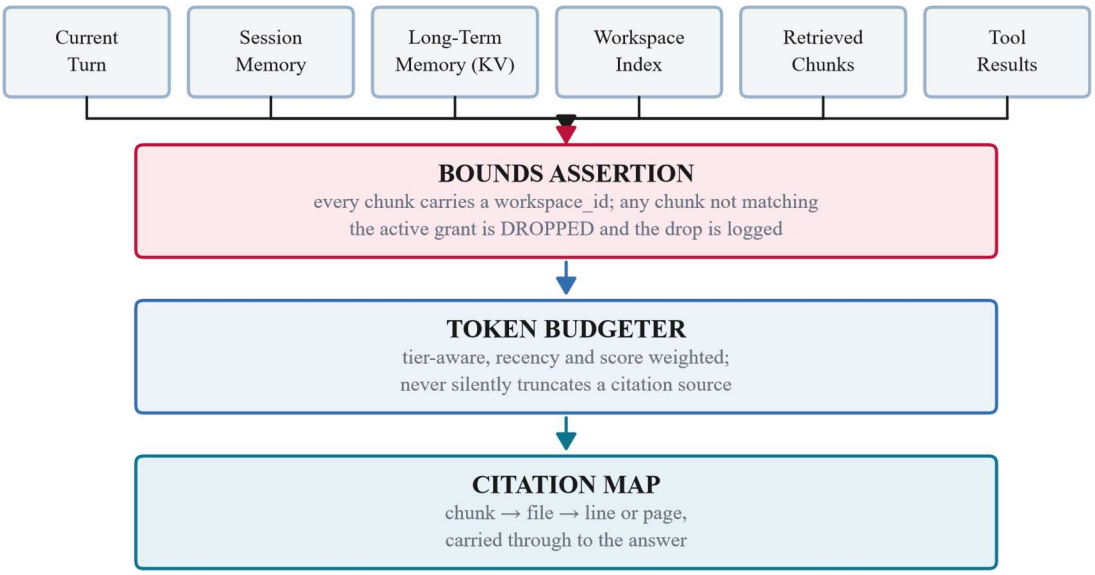


Figure 6.6: Context assembly.

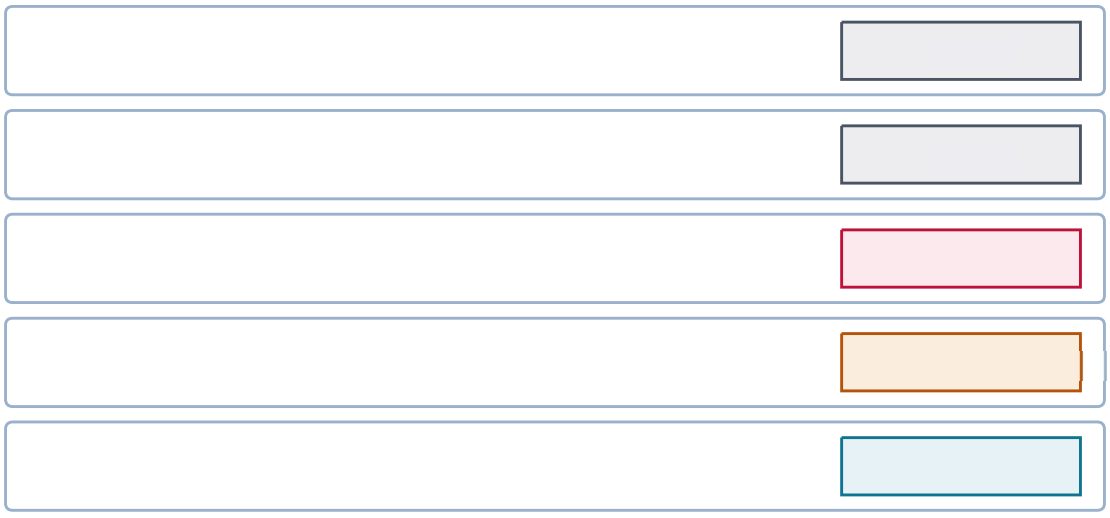


Figure 6.7: Policy evaluation order.

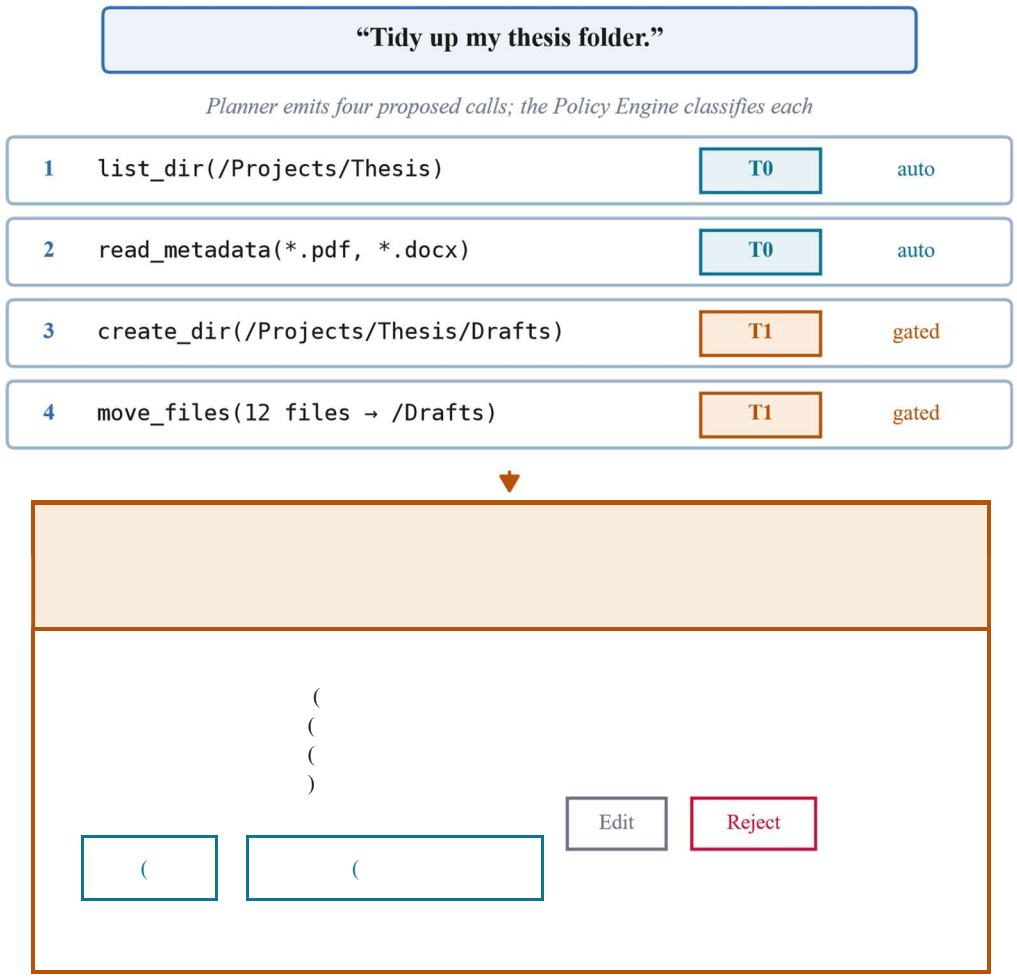


Figure 6.8: A worked classification for a folder tidy.

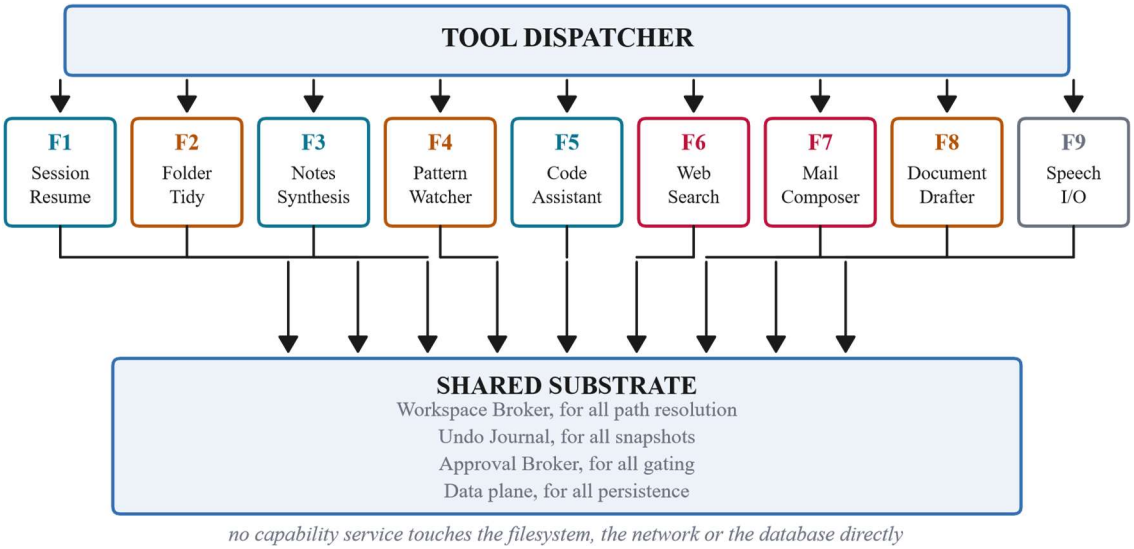


Figure 6.9: Capability services and their access path.

26 journal, the approval broker and the data plane as illustrated in Figure 6.9.

Capability services can only be accessed via the tool dispatcher and only reach the filesystem via the shared substrate, thus no service has its own path to the filesystem.

According to Table 6.2, the upper limit of each service can be requested. The table is a design constraint, not a description: a service cannot demand that it be served at a tier higher than it has stated maximum, which is what limits the impact of a defect in any feature..

Table 6.2: Capability services and risk tiers.

#	Service	Max tier	Notes
F1	Session resume	T0	Deterministic replay of recorded session state; no model involved
F2	Folder tidy	T1	Moves and groups only; no code path to deletion
F3	Notes synthesis	T0	Reads and generates; output remains a proposal until the drafter writes it
F4	Pattern watcher	T1	Detects three or more repetitions and may only propose a rule; cannot install one
F5	Code assistant	T0	Explains and produces a difference; never writes and never executes
F6	Web search	T2	On demand only; every query recorded

Table 6.2: continued from previous page

#	Service	Max tier	Notes
F7	Mail composer	T2	Draft, approve, send; sending is gated regardless of configuration
F8	Document drafter	T1	Always writes a new file; never overwrites an existing one
F9	Speech input and output	Not applicable	Not a service but a modality at layer two, over identical gates

The fact that the ninth feature is not a service is a structural assertion. Voice is normalised on the interaction layer into the same turn object as text, so it is automatically shared with all approving gate. Parity is obtained thus by construction and not by reimplementing the gates along a second path and the stage seven gate of Chapter 5 is provided to check this.

The logic that transforms a logical reference into a real path is the workspace broker, and Figure 6.10 presents five of its assertions. The second and third place inversion is the whole content of the guarantee: before the test is made, it is necessary to canonicalise a symbolic link or a segment of a traversal to the actual place where it belongs.

turnitin Page 21 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Canonicalisation precedes the containment test, of which a boundary and a suggestion differ.

6.4 Architectural Strategies

This section explains the strategies that influenced the architecture, the justification, and the alternatives that were thought of and discarded.

6.4.1.1 Local-First Operation, with Optional Egress.

The system is configured to use a local model, and cloud access is an opt-in per-workspace. The logic is tri-fold: the study documented local-first as the most popular deployment choice, privacy emerged as a risk of churn and a reoccurring free-text issue, and offline operation is a known limitation.

The other option of cloud-first functioning and a local backup was discarded as it reverses the guarantee. According to that arrangement the default behaviour is transmitting user data and the user has to take some action to avoid it, but according to the chosen arrangement the user has to take some action in order to allow it. The accepted trade-off is capability: a small local model is not as well as a large hosted model, which is alleviated through limited failover.

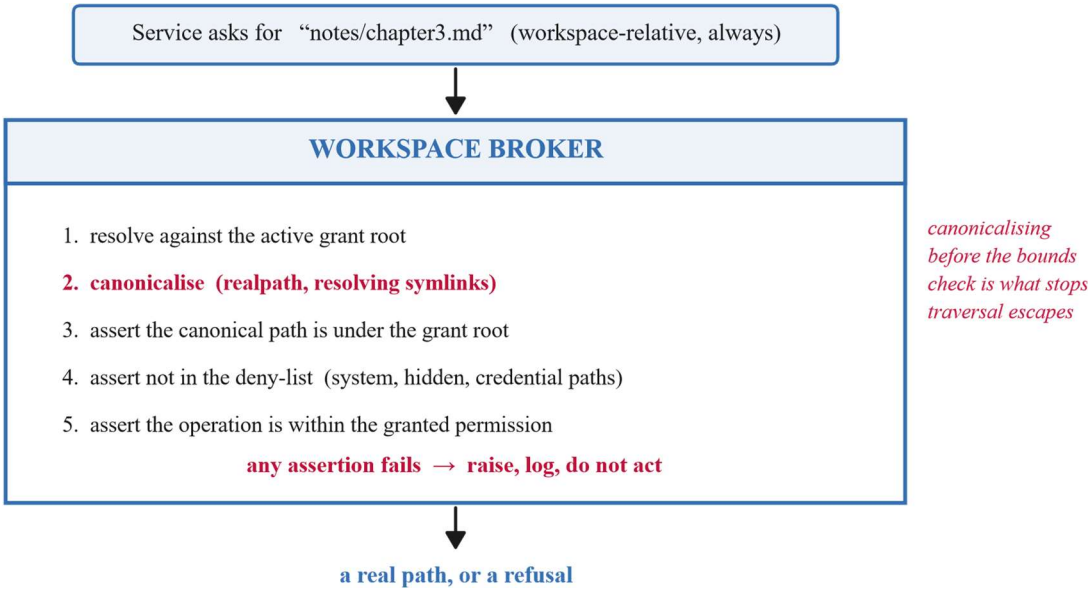


Figure 6.10: Path resolution and confinement.

where the workspace permits and by honest degradation where it does not.

turnitin Page 21 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

The degradation contract in Table 6.3 states the required behaviour as each external capability is withdrawn. Each row is a testable acceptance criterion rather than an aspiration, and together they define the system's correctness under adverse conditions.

Table 6.3: The degradation contract.

Withdrawn capability	Required behaviour
Internet connectivity	Full local operation continues. Cloud model calls, web search and mail sending are disabled, each failing as an explicit retryable action rather than being silently dropped
All cloud model providers	Falls back to the local model. Features exceeding local capability degrade to a stated limitation rather than an error
Local model	Falls forward to the cloud chain, if and only if the workspace permits cloud access
Remote device	The host continues uninterrupted. Approvals in flight revert to pending and appear on the host surface
Remote edge	Local operation is entirely unaffected, because the remote plane is not on the execution path

Table 6.3: continued from previous page

Withdrawn capability	Required behaviour
Workspace grant	The system cannot see the folder. Not declines to touch it: the path is not resolvable at all

6.4.2 Separation of Memory from the File Index

The system supports two types of knowledge and muddling the two is the fallacy that transforms a limited assistant into a spy device. Figure 6.11 illustrates the five types of memory and Figure 6.12 the difference that counts.

The kinds are differentiated by lifetime and by store, and all feed one context engine which combines, budgets and asserts bounds.

The two are answered to different questions and have different implications. The index is mechanical and rewritable; memory interpretive and, when erased, gone.

29 28 The file index provides an answer to what and where. It is mechanical, based on the filesystem and can be rebuilt at any time, and with no interpretation. Memory provides the answer of what the system has learned about a workspace, a task and the preferences of a user. It is interpretive, accumulated, lost when deleted and must be agreed upon to construct. The design makes them stay in different stores with different lifecycles, and each memory record is visible and removable individually in the

disclosure panel. And it is in this way that the most demanded unmet capability, that the system should learn the organisational habits of the user, is accommodated, without any extension of the boundary: such learning is per-workspace memory, and is subject to the same visibility and deletion as any other.

6.4.3 Disclosure as a Live Query

The disclosure panel is not what the system writes about itself but a live query on all the stores. The rationale is that a written summary may be out-of-date or wrong, but a query may not be in disagreement with the data it reads. All rows can be deleted and deletion is a true cascade, such that deleting a workspace not only deletes its index directory, but also its file rows, its embeddings and its memory entries.

The alternative, of a periodically generated report, was rejected since it would render the trust claim persuasive as opposed to falsifiable and a falsifiable claim is the superior position. The audit log is not intended to be deleted, as it is based on the principle of an audit log whose subject matter can be deleted is not an audit log..

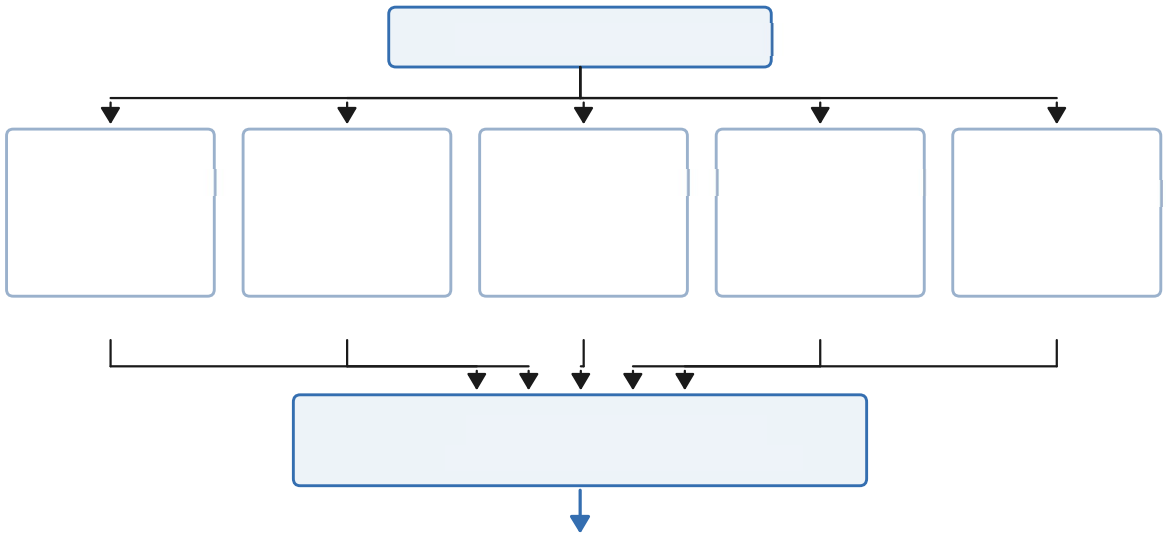


Figure 6.11: Five kinds of memory.

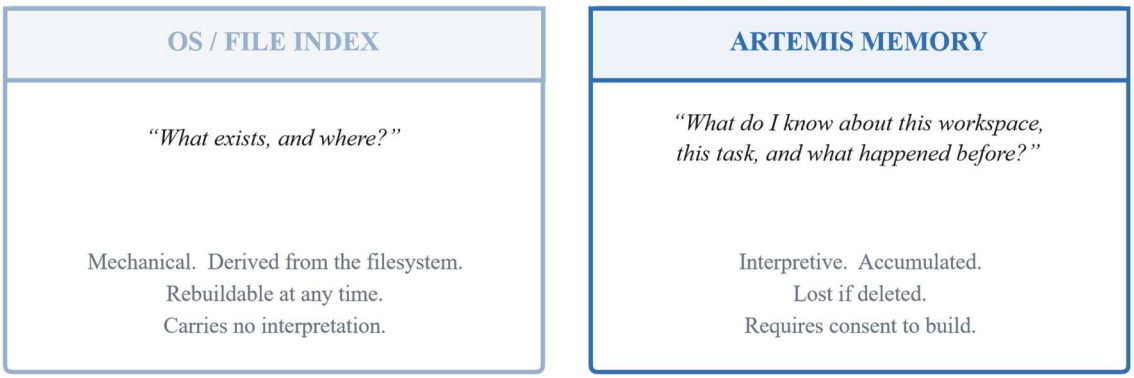


Figure 6.12: The file index compared with system memory.

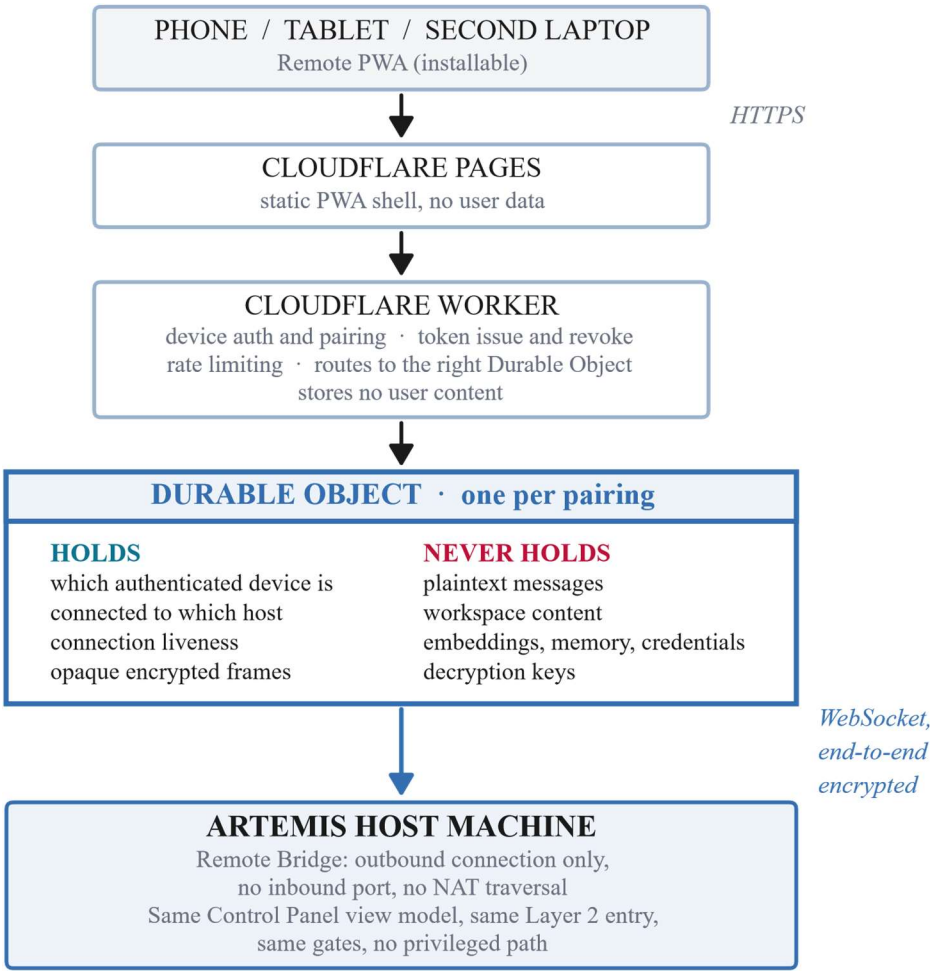


Figure 6.13: The remote access plane.

turnitin Page 25 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

6.4.4 The Remote Plane as a Detachable Component

The remote access plane is drawn outside the core stack, and the test of whether that separation is genuine is that removing the entire plane leaves a fully functional single-machine system. Figure 6.13 shows its structure and Figure 6.14 the pairing handshake.

The durable object records which authenticated device is connected to which host and relays encrypted frames it cannot read.

Host-side confirmation is deliberate, so that possession of the pairing code alone is never sufficient to pair a device.

The plane is a control surface rather than a privilege escalation. Remote sessions run at equal or lower privilege than local ones and never higher. A workspace may be marked local-only, rendering it invisible

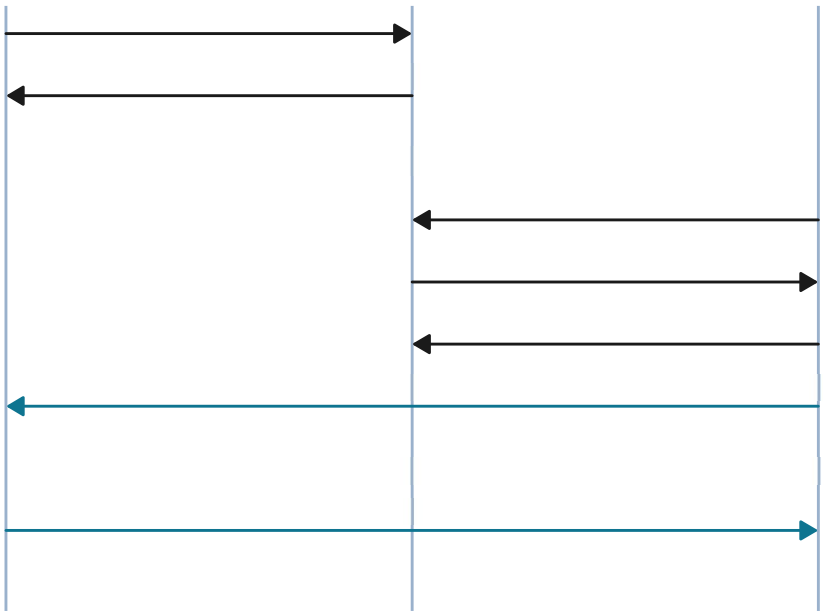


Figure 6.14: The pairing handshake.

to remote sessions. The outward actions can be set to need local approval irrespective of the location where the action is requested and this is the default. Sessions are time-based and on a host sleep, they are revoked by the host automatically and immediately, and all actions initiated by a remote user are recorded in the audit trail with their device. The relay does not possess plaintext of any type and thus the privacy assertion does not lose out on the edge compromise.

6.4.5 Asynchrony with Bounded Queues.

turnitin Page 25 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Figure 6.15 illustrates that the interactive path is thin and all that is expensive is asynchronous. The indexing is done in a dynamic fashion, with only those chunks whose content hash is different re-embedded, thus a single line edit re-embeds only a single chunk instead of a file. The two-stage retrieval is two-pass retrieval, where the inexpensive vector recall is followed by a reranking pass that keeps the index small and yet does not compromise on quality. The local model is detained with idle timer such that the initial token is not charged a cold load.

The interactive path does not block when on background work and background progress is reported visibly and is cancellable.

Each queue is limited and a full queue releases the lowest-priority work and claims to have done so instead of expanding indefinitely. The rationale is that unbounded queues turn a load problem into a memory exhaustion problem and that a system that silently falls behind is one the behaviour of which the user cannot predict.t.

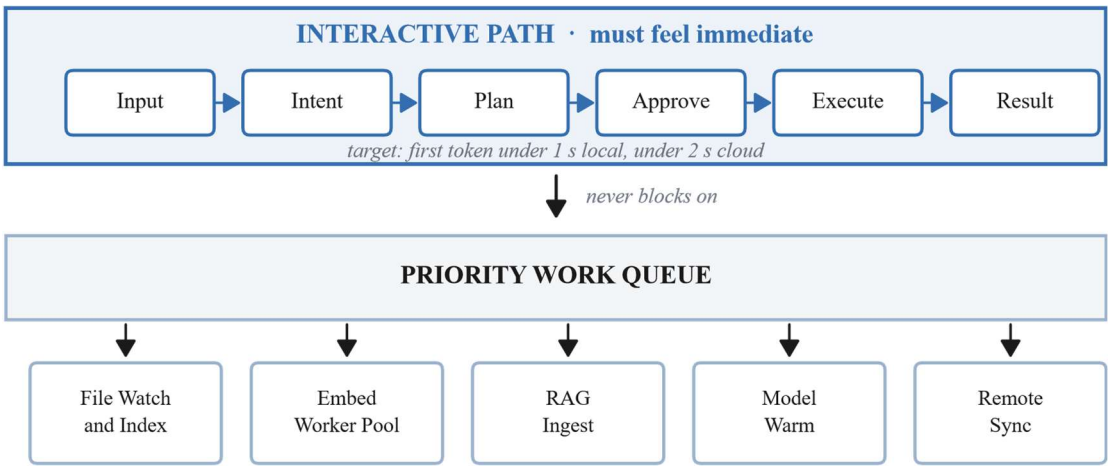


Figure 6.15: Asynchrony and background progress.

6.5 Domain Model/Class Diagram

The domain model will focus on four value objects that are immutable and five services that act on them. The value objects are not mutable in nature and hence a verdict cannot be attached to a call and the call manipulated before execution.

One of the proposed actions is called a ToolCall and contains an operation name, a workspace identifier, a list of paths relative to the workspace, arguments and a unique identifier. Its paths are relative, not absolute, such that an absolute path cannot be sneaked over the boundary by the loving care of its construction. It gives a description of previews and logs, and a fingerprint of the shape of the call regardless of what paths it crosses, which is the extent to which remembered approvals are used.

A Verdict is the end result of the policy engine, the decision on a single call, including the identification of the call it was applied to, the tier, the decision, the reason and the rule applied. It reveals the need of approval and whether the call was rejected, and can verify that it matches a particular call. It is that correspondence check, though, which prevents a call being a read and executed as a different kind of thing.

An ordered set of calls, having an intent, and an identifier is a Plan and the unit of approval and reversal.

A Resolved path is a path that has been asserted by all the assertions of the broker. This type is accepted by services and not a bare path so that an unchecked path does not accidentally access filesystem. To have one is the evidence that the line was crossed.

Services include the WorkspaceBroker, which addresses references or refuses; the PolicyEngine, which categorizes calls and remembers rules; the ApprovalBroker, which holds planned-to-be-done plans, constructs previews and records decisions; the UndoJournal, which records inverse operations and replays them; and the ToolDispatcher, which is the sole acting component.

This model has two relationships that bear the guarantees of the architecture. The dispatcher relies on the approval broker, instead of taking an approval flag provided by its caller, so that a service can not claim its authority. The dispatcher also solves paths via the broker in a second instance, although the policy engine has already done so, since the dispatcher is not allowed to rely on the previous component of the system having already executed its task correctly; that is what confused-deputy defects are about.

6.6 Policies and Tactics

This section specifies the policies and strategies that regulate implementation. They do not change the general structure, but outline the specifics of interface and implementation.

6.6.1 The Default Failure Mode, Refusal.

All the vague states culminate in denial. An unknown operation is rejected but not as a read. A dead end is not attempted, but is rejected. A call with a verdict that is missing or mismatched is rejected.

The execution can never be done by an expired approval and, that is why, the expiry is a unique result and not a type of rejection or silent rejection.

38 The other, explicit denial lists with permissive defaults, was discarded due to the fact that a denial list needs to know about all the dangerous operations whereas a permission list only needs to list the safe ones, and the result of an omission is of a different nature with the two.

6.6.2 Single Enforcement Points

Path resolution is carried out by the broker alone and execution by the dispatcher alone. Handler registration is an allowlist, which means that an operation that does not have a registered handler cannot be executed and the addition of a capability is an intentional action instead of an accidental result of a planner inventing a name.

One maintenance implication of this policy that must be declared to a future developer would be: adding a second path to the filesystem or a second point of execution would not be a refactoring but a change to what the system ensures, and would nullify the testing strategy, which relies on each.

turnitin Page 29 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

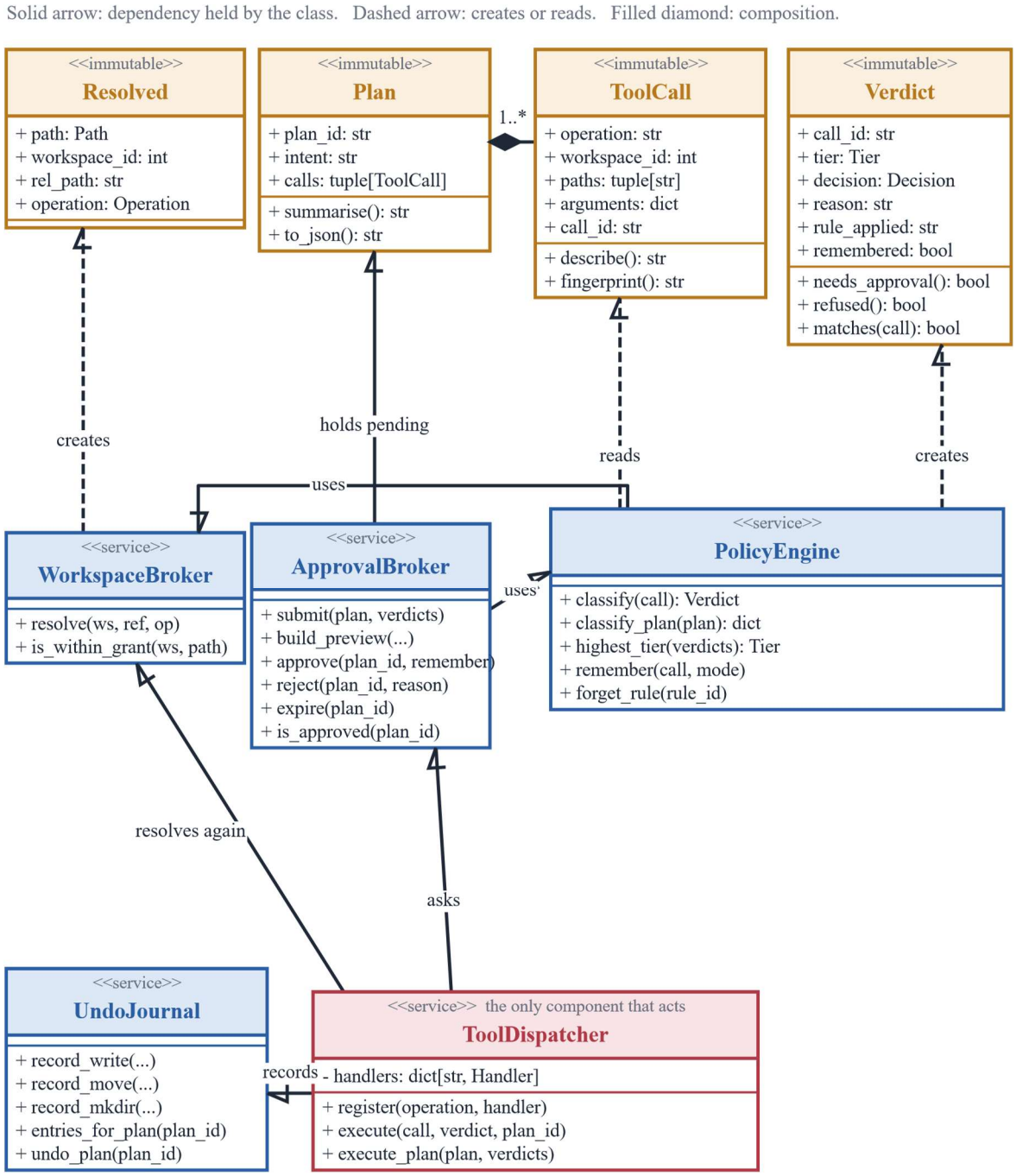


Figure 6.16: Class diagram.

guarantee having one place at which it can be verified.

6.6.3 Ordering Constraints

There are three load-bearing orderings which cannot be changed. The containment test is preceded by canonisation. The mutation is preceded by the snapshot. Policy review is followed by dispatch. All these are properties of correctness, not matters of taste, and turning any of them about results in a

turnitin Page 29 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

system that seems to work but offers none of the assurance it asserts. They are then each covered by a special test the aim of which is to fail when the ordering is perturbed.

6.6.4 Testing Strategy

The structure of testing is structured around the guarantees and not the code structure. The boundary is probed adversarially, with the escape techniques including relative traversal, symbolic links to outside the grant, absolute paths, and sibling directories bearing a common prefix in name, which all have to be rejected and logged. Each of the sets of operations, and the unrecognised case are exhaustively tested on policy classification. The properties that nothing is approved by default and outward operations cannot be remembered are tested. Recovery is checked by performing a plan, undoing it and comparing the outcome with the state that was previously stored.

The build sequence ends at each stage with a gate test and the end of a stage is when the gate succeeds and not when the code exists. Gates are designed in such a way that they can fail, the most obvious example being the speech stage gate that fails in case voice demanded any new approval gate.

6.6.5 Concurrency Handling

The interactive route is single-threaded but the store is accessed by over one thread, as the file watcher flushes its debounce buffer on a clock. The store thus serialises all its statements behind a lock, and all access is via its guarded means, as opposed to access to the connection itself. Disabling the thread guard on the database engine itself without this lock would trade a noticeable crash with an imperceptible corruption, thus the pair cannot be separated.

Chain read and append operations on the audit log are performed under one lock, and thus, two threads cannot chain on the same predecessor and bifurcate the chain.

6.6.6 Source Organisation

The source is structured according to architectural role and not by feature in order to have the correspondence between the design and the code direct. The fundamental elements, i.e., the broker, the policy engine, the approval broker, the undo journal, the dispatcher and the disclosure panel, are contained in a single package. Another plane is the data plane. The surfaces of interfaces are in a third place. A reader in need of the enforcement point of a guarantee can find it where the architecture claims it is.

6.7 Conclusion

This chapter has introduced the system design: the layers and their components, the strategies that shape the architecture, domain model and the policies that bind the design. Each significant decision

was traced to a requirement or a research finding. The design puts the path resolution, classification and execution in one component, and the remote plane is detachable so that the guarantees which the hypothesis relies on are structural properties and can be tested as such. The following chapter details the prototype which realizes this design and tests which are exercising the design..

turnitin Page 32 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Chapter 7 Implementation and Test Cases

This chapter explains the prototype that was constructed throughout the initial phase of the project and the tests that exercise this prototype. Since the first deliverable is possible in the template, the implementation section will be a description of the prototype and not of a complete product. It gives the platform and interfaces used, the algorithms behind the components that are not apparent from the design, and the state of each stage of the build sequence. In the second part the design of the test cases is established and a representative set is given in the specified layout, each follow-up to the requirements of Chapter 4. The third section presents the test measurements and requirements traceability to tests. The chapter does not copy the source code, but documents the processes that the code executes..

7.1 Implementation

7.1.1 Scope and Status of the Prototype

The prototype is a working desktop assistant written in Python 3.13. It comprises 56 modules and about 14,250 lines of source, organised by architectural role into five packages as Chapter 6 describes: the core enforcement layer (26 modules, about 5,400 lines), the local data plane (4 modules, about 850 lines), the capability services (10 modules, about 2,100 lines), the presentation surfaces (5 modules, about 4,450 lines) and the optional remote plane (10 modules, about 1,450 lines). A further relay program in JavaScript and a phone page complete the remote plane. It is accompanied by an automated suite of 554 tests.

All eight stages of the build sequence of Table 5.2 have been implemented, each ending in the gate test that Chapter 5 requires. Table 7.1 records the state of each stage. A stage counts as having its gate passing when the automated suite passes. It counts as validated through the interface only where the user-interface campaign of Chapter 9 supports it, and that campaign found a number of defects that the automated gates did not, so the two columns are reported separately.

Table 7.1: Implementation status by stage.

Stage	Deliverable	Automated gate	Interface campaign (Chapter 9)
1	Workspace foundation	Passing	Gate partly met; folder guard-rails missing

turnitin Page 32 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

2	Policy, approval and undo	Passing	Gate partly met; undo defect on name collisions

Table 7.1: continued from previous page

Stage	Deliverable	Automated gate	Interface campaign (Chapter 9)
3	Gateway: router, context, planner, dispatcher	Passing	Gate met for execution; one retrieval isolation failure
4	Resume, tidy, repeated tasks	Passing	Gate partly met
5	Notes synthesis and code assistance	Passing	Gate not met: unsupported claims not always labelled
6	Web search, mail, drafting	Passing	Gate met; visibility of what left the computer weak
7	Speech input and output	Passing	Gate met (no new approval gate); recording control defect
8	Remote access plane	Passing	Not fully tested; one boundary defect

7.1.2 Platform and Interfaces

16

Table 7.2 lists the platform, libraries and services on which the prototype depends and the role of each. Every dependency of the core is local. The network is used only by the optional cloud providers, the web search service, the mail account and the remote plane, each behind its own gate.

Table 7.2: Platform, libraries and services.

Component	Role
Python 3.13, pytest	Implementation language and test runner (554 tests).
SQLite [3]	The system of record: workspaces, file index, audit log, undo journal, chat sessions, settings; one vector index database per workspace.
Ollama [2], model gemma4:e4b	Default local language model, reached over the loopback interface; model name is a user setting.
nomic-embed-text through Ollama	Local text embedder for the vector index.

Cloud providers (Gemini, OpenAI, Anthropic)	Optional fallback tiers, available only to workspaces that opted in.

Table 7.2: continued from previous page

Component	Role
LangGraph [1]	State-graph runtime for the planner, with an explicit pause before any gated call.
NumPy	Cosine similarity over a workspace's embedding matrix.
watchdog	File watcher feeding incremental indexing.
Gradio 6.12	The web interface with tabs Work, Folders, What it knows and Settings.
PySide6, Typer	A desktop panel and a command-line surface over the same components.
faster-whisper	Local speech recognition.
Kokoro, Windows Piper, speech	Speech synthesis, tried in that order.
cryptography, keyring	X25519, HKDF and AES-256-GCM for the remote plane; storage of keys and passwords in the operating system keyring.
Cloudflare Worker with a Durable Object	The deployed relay of the remote plane and the host of the phone page.

7.1.3 Implementation of the Workspace Broker

A path is only created when the only component that transforms a reference into a path is the broker. It implements five claims in a given order: the reference is resolved against the root grant; it is canonicalised, the symbolic links and parent-directory elements are substituted by the actual location; the canonical path is checked to be contained within the grant root; it is checked against a denylist; and the requested operation is checked against the permission of the grant. The canonicalisation comes before the containment test in that a symbolic link in the grant which has its target out of the grant appears contained until it is resolved. Credential and system files and locations, including .ssh, .aws, .env and key files, are always included in the denylist, and the store of the application itself is always included, so that a grant on a parent folder does not reveal the audit log. An effective solution provides a value that documents the workstation, the relative path and the operation. Only this value

can be used by services and not a bare path, so an uncontrolled path cannot accidentally reach the file system.

Each refusal is documented to the audit log prior to the refusal being raised.

7.1.4. Implementation of Policy Engine and Approval Broker.

A proposed call is all the policy engine reads. It does not even look at the text of the model, so whatever is written in the inner part of any document cannot be able to defend itself to a new decision. The evaluation is executed in a fixed order, i.e. the grant check, the destructive check, the egress check, the mutation check and lastly allow, the first rule that matches is the deciding one. The call is made in only one tier. No approval of reads. Changes made are reversible except when the user has selected to permit that operation without question in that workspace. Actions towards the outside are never kept and cannot ever be recalled. Unrecognised destructive operations are denied a permission course and an unrecognised operation is denied also, such that nothing omitted can ever broaden the permission. The top level of the steps of a plan is the top level. Classification never makes any mistake: a call that is not classifiable is a refusal.

The approval broker keeps each plan in one of four states, being pending, approved, rejected and expired, and none of the transitions out of pending is automatic. One plan may not be voted twice, expiry is not a form of approval and an approved plan is scoped to a single operation within a single workspace. The preview displays the workspace, the suggested operations, the level, whether something leaves the computer and whether something is deleted and in outward steps displays the specific search text or message body which the user approves of what actually will be done.

7.1.5 Introduction of the Dispatcher and the Undo Journal.

The only component that operates is the dispatcher. It refuses each call in order: a call having no verdict is refused; a verdict delivered in respect of a different call is refused, since each verdict is marked by the identifier of the call; a denied call is never executed; a gated call is refused unless the approval broker has a recorded approval to its plan. The caller never passes the approval but looks up the approval. The dispatcher also finds the paths of the call by the execution time via the broker, hence, the boundary is even when a previous layer was skipped.

The undo journal documents the method of undoing a change prior to the change. A copy of a small file is moved into a content-addressed blob store: thus, the same data is stored only once, and a file larger than four mebibytes is hard-linked where the file system allows. A motion or a newly generated file does not require a copy since it is inverse structural. Undo operates at a plan-granularity, undoing all the recorded steps in reverse order, and leaving a folder, filled in by the user in the meantime. Both the dispatcher and the journal write to an append-only audit log where each entry has the hash of the last entry such that deleting or modifying an entry will cause the chain to break, and on-demand the chain can be checked. Chain read and append are actions performed under a single lock thereby preventing forking of the chain by two threads.

7.1.6 Introduction of the Gateway.

The typed request is first sent through an intent step which determines whether the request is a question, a file task, a draft, a search, a mail request or a request about code. It is rule-based, with patterns applied to the request text so routing is deterministic, quick and verifiable and does not rely upon the opinion of a model. The model router then chooses a provider. Providers are retried in tiers order, starting with local ones, and with a single retry per tier, so that a dead chain will fall over in seconds and with a failure notice. Egress is also determined by the router which is the only location to determine a workspace that has not subscribed to cloud has an exclusive tier to the workspace and when local model is inaccessible the response is denial and not silent upgrading to a cloud provider.

The context engine puts together what the model can see. Every fragment has its provenance, is the text of the user, a workspace file, a tool output, memory or the web. The words of the user are the only authority, all other sources are displayed to the model as data. The active workspace limit retrieval and the context size is budgeted with an estimate of 3.6 characters per token. The planner is a LangGraph state graph, which requests the model to give a structured plan of a low sampling temperature of 0.1, verifies it against the plannable operations and gives up to three planning cycles with a time budget of 30 seconds, and plans, but does not execute. The model can thus suggest, but has no handle by which it can act.

7.1.7 Introduction of Indexing, Retrieval and Grounding.

The files within a workspace are partitioned into slices of not more than 800 characters with its source file and line range so a citation can be made referring to it. The local embedder normalizes each chunk, and stores it as a float32 vector in a workspace-private database. A search inserts the query, and

calculates the cosine similarity with the matrix in the workspace in a single product of matrices, returning the optimal chunks. The indexing is incremental: a file that has not changed in content is not re-embedded, files larger than two mebibytes are not indexed, and an indexing worker does not block the interactive path. Changes are signalled to the index using a file watcher that has a debounce buffer.

A grounding gate is put on each response to a question regarding the files of the user. The answer is divided into claims, each in a sentence and a claim is justified when it has a citation that identifies a chunk sent to the model and is a file chunk and not the words of the user. Any other claim is indicated with an unsubstantiated label. The gate provides that there is a source and that a source has been given and does not provide that the source states what the claim states that it is a restriction which is inherited by the literature and recorded as such in Chapter 3.

7.1.8 Implementation of the Capability Services

Only the dispatcher and the broker can access the file system through all services. When you resume, the files that were last in a workspace are restored along with a note that was left by the user to resume. Arrange loose files in a folder by type, with extension tables of documents, spreadsheets, pictures and others, or by date as required. Before making the moves, it preplans the creation of each destination folder and leaves behind a file that has no counterpart of its type and files that have already been filed in subfolders, assigns a conflicting name a different name rather than an overwrite, and displays every move in the preview. The service has no deletion operation to make a call. Repeated-task detection calculates a signature on each approved plan, and ignores undone plans and plans the user has rejected, and provides an automation when the signature has been repeated at least three times over a 30-day window. Code assistance finds responses to code questions and presents any proposed change as a difference, and nothing in the service writes to a file. Web search only operates on an explicit query, via a public search endpoint, displaying the query in the preview, logs all queries, and labels results as web searches. Mail only sends a draft using the mail account of the user, after the draft is approved; the recipient is authenticated, header injection is not allowed and the recipient and time of each send are logged. Drafting creates a new file whose name will not conflict with any existing file and will never replace it; undo deletes it.

7.1.9 Implementation of Disclosure

The panel with the title What ARTEMIS knows is a live query of the store and is not a cached list. It lists every workspace, every indexed file, the hidden chunks, remembered facts, retained chats,

automation rules, policy exceptions and the history of outward actions, and it also has a deletion of each item, with a confirmation, other than the audit log. Deleting a workspace removes the index entries, embeddings, sessions, memory and rules based on it and leaves files owned by the user unaffected.

7.1.10 Implementation of Speech

Speech is an interaction layer modality but not a service. Audio will also be accepted as long as a voice session is armed by the user. Faster-whisper transcribes a recording on the computer, which is deleted once read and even when transcribing fails. The audit trail documents that it was voice and not what was said. The transcript is sent to precisely the same point of entry as the typed request thus a spoken request is subjected to the same gates and a spoken email is awaited approval like a typed one. Replies will be read out by the first of three engines which will be tried in the order of naturalness. In front of speech, the reply is deprived of the model label and of markup and is trimmed, and a spoken chat is phrased as speech. The hands-free loop is contained in the browser: once it has listened to a reply, it considers a silence of approximately 3.5 seconds after the user has spoken as the end of the user turn, and, when the user takes over four seconds to say anything, it speaks first and then resumes listening.

7.1.11 Introduction of the Remote Plane.

The distant airplane allows a matched phone to communicate with the computer using a relay that transports opaque frames with no other knowledge. An exchange of X25519 keys where the short pairing code is concatenated into the derived key are done to pair the devices such that a relay that replaces keys learns nothing, and the owner of the device confirms on the computer after comparing a short safety number. AES-256-GCM is used to seal frames with a per-session key and counters, such that a replayed frame will not be answered twice. The key of the phone is stored as a non-extractable key in the browser and the key of the host in keyring of the operating system. One hour is one session which then has to be opened again. The bridge reveals asking and answering only: folders are not visible on a device unless they are ticked, no outward approvals can be received on a device, it must provide on the computer, voice on a device is default, and each remote request is logged against its device. Revocation of a device is immediate. In voice, the phone transmits small blocks of 16 kHz audio over the encrypted channel, the computer determines when the turn is over by counting audio time and the response is sent back sentence by sentence as text and speech. The plane is optional in nature, in that nothing in the core brings in the remote package, and the main database contains no remote

turnitin Page 39 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

tables. The relay is implemented as a Cloudflare Worker with a Durable Object serving the phone page, and a runnable relay with the same rules is made available to use on a home network and to test.

7.1.12 User Interface implementation.

There are four areas in the web interface. The request box, the folder selector, the approval cards and the saved chats, which storage lasts seven days, are stored in work, along with the voice controls. Workspaces are added and listed by typed path by Folders. The disclosure panel is what it knows. The settings contain the model options, the cloud policy per folder, the mail account and the remote access controls. Gradio 6.12 limits interaction somewhat, and the interface operates within those limits by utilizing hidden controls which are invoked by scripts in the page. The identical elements can be accessed by a command line and a desktop panel.

7.2 Design and description Test case.

The tests get structured on the guarantees of Chapter 4 and not on the code structure. The following are their common attributes. Each test is run against a throwaway store, which has been chosen by an environment variable, and against a throwaway folder which represents the files of the user, such that none of the tests touch real data, and tests do not interfere with one another. No test requires the network, a cloud key or a running language model: model responses are provided by stub providers, embeddings by a deterministic fake embedder, mail and search by stubs, and speech recognition and synthesis by stand-ins. The suite thus verifies the enforcement layer and the plumbing deterministically and is unable to verify the quality of what a real model claims. Tested Gate tests are written in such a way that they fail, in order to test a property an attempt at breaking the gate: escaping the boundary, an unknown operation to the policy engine, a call with an unknown verdict to the dispatcher, and a requirement of any new approval path in the case of speech. Every test case below indicates the tests it summarises and the tables are in the format required by the template..

7.2.1 Test Cases

The nine test cases below are TC-01 to TC-09, in Tables 7.3 to 7.11.

Table 7.3: Test case TC-01: workspace boundary.

Workspace Broker
Requirements FR-2, FR-3, FR-4; stage gate of the workspace foundation stage (tests/test broker.py, 13 tests)

turnitin Page 39 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Test Case ID: TC-01		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): UC-1, UC-3
Revision History:	None; first version.	
Objective:	Establish that no reference can resolve to a path outside an active grant, that every attempt is recorded, and that the record is intact.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer. The broker is the single component that turns a reference into a path.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	The machine can create symbolic links (checked at run time; the symbolic link case is skipped otherwise).	
Pre-Requisite:	A folder with two files is granted as a workspace; a second folder holding a file named secrets.txt exists beside it and is not granted.	
Step No.	Execution description	Procedure result
1	Resolve readme.txt inside the grant.	A resolved path is returned, with the workspace identifier and the relative path.

Table 7.3: continued from previous page

2	Resolve references that climb out of the grant with parent-directory segments.	A boundary refusal is raised with the reason outside grant.
3	Place a symbolic link inside the grant that points at the ungranted folder, then resolve a file through it.	Refused. The link looks contained; only its canonical target is not, which is what canonicalising first exists to catch.
4	Resolve an absolute path and a sibling folder whose name begins with the grant's name.	Both refused; a shared name prefix is not containment.
5	Read the audit log after a refused attempt.	One refused entry exists for the attempt and contains the reference text; the hash chain verifies.
Comments: All 13 tests passed, including the symbolic link case on the test machine.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.4: Test case TC-02: policy engine classification.

Policy Engine		
Requirements FR-7, FR-8, FR-9, FR-10, FR-11 (tests/test policy.py, 17 tests)		
Test Case ID: TC-02		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): UC-2, UC-3
Revision History:	None; first version.	
Objective:	Show that the four tiers behave as specified and that the destructive tier has no approval path.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer. Classification reads the tool call only and never the model's reasoning.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Remembered approvals are off by default.	
Pre-Requisite:	A folder is granted as a workspace.	
Step No.	Execution description	Procedure result
1	Classify a read of a file inside the workspace.	Allowed to run without asking.

Table 7.4: continued from previous page

2	Classify a change to a file and a call that sends something out of the computer.	Both held for approval; the second is flagged as leaving the computer.
3	Classify a delete operation.	Tier destructive, decision refuse. No approval can change this.
4	Classify an operation the engine has never heard of (rm rf).	Refused under the rule unknown-operation, not assumed safe.
5	Classify a read whose path escapes the grant.	Refused under grant-check, which runs before every other rule.
6	Ask to remember an outward action and a destructive action.	Both remain gated or refused; remembering is scoped to one operation in one workspace.
Comments: All 17 tests passed.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.5: Test case TC-03: tool dispatcher.

Tool Dispatcher		
Requirements FR-17, FR-35; invariants of Chapter 6 (tests/test dispatch.py, 11 tests)		
Test Case ID: TC-03		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): UC-2
Revision History:	None; first version.	
Objective:	Show that the dispatcher cannot be talked into acting without a verdict and an approval, and that a full plan can be gated, executed and undone.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer. The dispatcher is the only component that acts.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Handlers are registered by the services; unregistered operations have none.	
Pre-Requisite:	A granted workspace with a file readme.txt containing the word hello.	
Step No.	Execution description	Procedure result
1	Execute a read call with no verdict.	Refused: no policy verdict.

Table 7.5: continued from previous page

2	Execute a change to readme.txt after classification but without an approval for its plan.	Refused: needs approval. The file still contains hello.
3	Execute a call the policy denied, and a call for an operation with no handler.	Neither executes.
4	Execute a call whose path escapes the workspace, with a verdict that claims otherwise.	Refused again at execution, so the boundary holds even if an earlier layer is bypassed.
5	Run a full plan: classify, submit, approve, execute, undo.	Gated, executed once, snapshotted before the change, and restored by undo; every refusal reaches the audit log.
Comments: All 11 tests passed.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.6: Test case TC-04: undo of an approved plan.

Undo Journal		
Requirements FR-15, FR-16 (tests/test undo.py, 10 tests)		
Test Case ID: TC-04		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): UC-2
Revision History:	None; first version.	
Objective:	Show that recovery information is recorded before a change and that undo restores the state the user approved from.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer. Snapshots are kept in a local blob store; moves need no copy.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Files are below the hardlink threshold unless stated.	
Pre-Requisite:	A granted workspace with two files, readme.txt and notes/chapter3.md.	
Step No.	Execution description	Procedure result

Table 7.6: continued from previous page

1	Record the creation of a Drafts folder and the moves of both files into it, performing each change after its record is written.	Both files sit in Drafts; the originals are gone.
2	Undo the plan by its identifier.	Three steps reversed, no problems reported.
3	Inspect the workspace.	Both files are back at their original paths and the created folder is removed.
4	Undo a plan after the user has put a new file into the created folder.	The folder is kept, because removing it would destroy the user's file.
5	Undo an unknown plan, and undo the same plan twice.	Reported, not raised; a reversed plan is cleared from the journal; each undo is written to the audit log.
Comments: All 10 tests passed. The campaign of Section 9.4 later found a name-collision case that these tests do not cover.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.7: Test case TC-05: approval broker.

Approval Broker		
Requirements FR-12, FR-13, FR-14, FR-17 (tests/test approval.py, 12 tests)		
Test Case ID: TC-05		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): UC-2, UC-4
Revision History:	None; first version.	
Objective:	Show that a pending plan stays pending until a person decides, and that the preview tells the truth.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	None.	
Pre-Requisite:	A tidy plan with several moves, and a plan containing an email.	
Step No.	Execution description	Procedure result

Table 7.7: continued from previous page

1	Submit the plan.	State is pending, and remains pending without further action.
2	Read the preview.	It states what will and will not happen, and flags anything that leaves the machine.
3	Reject the plan; separately let another plan expire.	Neither becomes approved; the stored outcomes are rejected and expired.
4	Decide a plan a second time.	Refused; a plan cannot be decided twice.
5	Approve and remember a plan that contains an outward step.	Only the gated reversible calls are remembered; the outward one is refused for remembering.
Comments: All 12 tests passed.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.8: Test case TC-06: grounding gate.

Grounding gate
Requirements FR-22, FR-23 (tests/test grounding.py, 7 tests)

Test Case ID: TC-06		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): Notes synthesis (Chapter 4, feature F3)
Revision History:	None; first version.	
Objective:	Show that the gate marks any claim that cannot be traced to a chunk of the user's own files that was actually sent to the model.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, core layer, applied to model output before display.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Citations take the form [n], where n numbers a chunk in the context.	
Pre-Requisite:	A context of three chunks: the user's question, report.md line 4, and a second file.	
Step No.	Execution description	Procedure result
1	Check the claim Revenue grew 10% [2].	Supported, with the source report.md:4.
2	Check a claim with no citation.	Unsupported.

Table 7.8: continued from previous page

3	Check a claim citing a chunk number that was never sent.	Unsupported.
4	Check a claim citing the user's own question as its source.	Unsupported; the user's words are not evidence.
5	Render a two-sentence answer with one unsupported sentence.	Only that sentence is prefixed with the label [unsupported].
Comments: All 7 tests passed. The gate checks that a citation exists and points at a sent chunk; it does not check that the chunk says what the claim says (an inherited limitation, Chapter 3).		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.9: Test case TC-07: outward actions.

Outward actions: mail, search and drafting
Requirements FR-10, FR-25, FR-26, FR-27 (tests/test sprint7 gate.py, 24 tests; tests/test mail.py, 11 tests)

Test Case ID: TC-07		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): UC-4
Revision History:	None; first version.	
Objective:	Show that sending mail and searching the web are always gated and recorded, and that drafting never overwrites.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, services layer, over the same dispatcher.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	A stub mail server and a stub search provider replace the network.	
Pre-Requisite:	A granted workspace and a stubbed mail account.	
Step No.	Execution description	Procedure result
1	Compose a message.	Nothing is sent; a draft is shown.
2	Classify the send and the search.	Both gated as outward.
3	Approve and remember a send.	Refused: an outward approval can never be remembered.
4	Reject the mail and then dispatch it directly.	Never sent, even when dispatched directly.

Table 7.9: continued from previous page

5	Approve and send; then read the log.	Sent once; the recipient and time are recorded.
6	Draft a document whose name already exists, and one whose name appeared after planning.	The drafter picks a new name; the existing file is untouched; undo removes the new file.
Comments: All 35 tests passed. Header injection in the subject and a malformed recipient are refused.		
Passed [X] Failed [ ] Not Executed [ ]		

Table 7.10: Test case TC-08: remote device limits.

Remote plane: bridge and plane
Remote access stage gate (tests/test remote.py, 20 tests; tests/test remote network.py, 2; tests/test remote voice.py, 9)

Test Case ID: TC-08		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): Remote access (Chapter 6, Section on the remote plane)
Revision History:	None; first version.	
Objective:	Show that remote access is optional, encrypted end to end, and strictly below local access.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, remote package. Nothing in the core imports it.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	A local relay (in-memory or a real WebSocket on the loopback interface) stands in for the deployed edge.	
Pre-Requisite:	A workspace is granted; a device is paired and confirmed on the computer.	
Step No.	Execution description	Procedure result
1	Pair without the owner's confirmation; pair with a wrong code.	Not paired; repeated wrong codes lock the pairing.
2	Capture every frame the relay carries.	No plaintext appears; a replayed frame is not answered twice.

Table 7.10: continued from previous page

3	Ask for a change from the device.	A plan is held; nothing changed on disk until the device approves, and then only the approved change occurs.
4	Ask for an email and approve it from the device.	The answer is waiting on the computer; the plan is still pending locally.
5	Send settings, forget key, add folder, erase and set model requests.	Every one returns an error.
6	Revoke the device, and, separately, delete the remote package and start ARTEMIS.	The device is cut off at once and for good; ARTEMIS runs unchanged.

Comments: All 31 tests passed. The campaign of Section 9.4 found a case this suite missed: a waiting plan still ran after its folder was un-shared.
Passed [X] Failed [ ] Not Executed [ ]

Table 7.11: Test case TC-09: speech and the approval gates.

Speech input and output		
Requirements FR-28, FR-29 (tests/test speech.py, 14 tests)		
Test Case ID: TC-09		QA Test Engineer: Project team (automated suite)
Test case Version: 1.0		Reviewed By: Project team
Test Date: 9 November 2026		Use Case Reference(s): UC-2, UC-4
Revision History:	None; first version.	
Objective:	Show that voice is another way to type: it is never recorded unless armed, and it cannot approve, send or delete.	
Product/Ver/ Module:	A.R.T.E.M.I.S. prototype, services layer. Speech engines are replaced by stand-ins for the test.	
Environment:	Windows 10 Pro, Python 3.13, pytest. Each test runs against a throwaway store (ARTEMIS HOME set to a temporary folder) and a throwaway folder standing in for the user's files. No network, no cloud key and no language model are needed.	
Assumptions:	Transcription and speech synthesis are stubbed.	
Pre-Requisite:	A granted workspace and a voice session in its default, disarmed state.	
Step No.	Execution description	Procedure result
1	Offer audio to a session that has not been armed.	Refused unread.

Table 7.11: continued from previous page

2	Arm the session, read a recording, then disarm.	Listening stops; the recording file is deleted after it is read, and also when transcription fails.
3	Speak a request to move a file.	A plan is held for approval exactly as for a typed request.
4	Speak a request to email someone.	A draft waits for approval; nothing is sent.
5	Read the audit log.	It records that voice was used and never what was said.
Comments: All 14 tests passed. These tests use stand-in engines, so they establish the gates and not the quality of recognition or speech; the campaign testers could not hear audio either.		

turnitin Page 49 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Passed [X] Failed [ ] Not Executed [ ]

7.3 Test Metrics

Table 7.12 gives the test metrics for the automated suite at the end of the first phase, using the metrics prescribed by the template. The suite was executed in full on 9 November 2026 and took 42 seconds.

Table 7.12: Test metrics.

Metric	Value	Basis
Number of test cases	554	Collected from 39 test files, one file per component or stage gate.
Number of test cases passed	554	Full run of 9 November 2026.
Number of test cases failed	0	Same run.
Test case defect density	0%	Failed multiplied by 100, divided by executed: 0→100/554.
Test case effectiveness	0% against interface defects	Defects detected by the test cases multiplied by 100, divided by total defects detected, taken over the 71 distinct defects of the interface campaign: all were found while the automated suite was green, so 0 100/71. See the discussion below.

The first three figures describe a suite that passes completely, and the fourth shows why that is not sufficient. The defects that the later interface campaigns found were not detected by the automated tests, because those tests checked each component in isolation and the defects were in the way components are joined to the interface. In the first hands-on round, 359 unit tests were passing while only one of nine features worked end to end. The effectiveness metric is therefore reported as it is. It is computed over the interface defects only, because the defects that the unit tests caught during development were not logged separately. The lesson drawn from the figure is a practice that continues into the second phase: every defect found at the interface is to be given a regression test before it is closed.

7.3.1 Traceability Matrix

Table 7.13 traces each group of requirements of Chapter 4 to the test files that verify it, and to the result of the interface campaign. Each group of requirements from FR-1 to FR-35 has automated tests.

turnitin Page 49 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

The audit requirement FR-18 has no file of its own because it is asserted inside the tests of the broker, the policy engine, the approval broker, the dispatcher and the undo journal, each of which checks that its refusals or actions reach the log. The remote plane, which carries no numbered functional requirement, is verified by 31 tests.

Table 7.13: Requirements traceability.

Requirement group	FRs	Verifying tests (count)	Campaign
Workspace grant, boundary, index	1 to 6	broker (13), workspace and index (13), watcher (5), vector index (11)	Partial
Classification, approval, execution	7 to 14, 17, 35	policy (17), approval (12), dispatch (11), gateway (11), planner (23)	Partial
Recovery	15, 16	undo (10)	Partial
Audit	18	Assertions within the five files above	Partial
Resume, tidy, repeated tasks	19 to 21	resume (17), tidy (12), patterns (16)	Partial
Synthesis, grounding, code	22 to 24	grounding (7), answer (6), chunking (9), code diff (11)	Fail on the gate
Outward actions	25 to 27	sprint 7 gate (24), mail (11)	Pass
Speech	28, 29	speech (14), remote voice (9)	Fail on recording control
Disclosure	30, 31	panel (18)	Partial

Table 7.13: continued from previous page

Requirement group	FRs	Verifying tests (count)	Campaign
Model access and context	32 to 34	router (18), cloud policy (3), providers (2), model configuration (29), context (10)	Partial

Interface and conversation (not numbered)	–	intent (39), chat (34), assistant (32 and 15), web (29), capabilities (10), credentials (12), others	Partial

7.4 Conclusion

The prototype implements all eight stages of the build sequence and its enforcement layer behaves as specified under 554 automated tests, none of which fails. The implementation follows the design in the respects that carry the hypothesis: path resolution occurs in one component, classification in one, and execution in one, and the model proposes without acting. The test design is adversarial where the guarantees are concerned and is isolated from the network and the model, which makes it fast and repeatable and also limits it, since it cannot judge what a real model says or how the interface behaves under use. The metrics show this limit candidly. The gaps between a green suite and the defects of the interface are the subject of the next two chapters.

Chapter 8 User Manual

The user manual will be added in FYP-II, as the template provides. The prototype can be started and used today from the instructions in the repository, and the running instructions and a description of the four areas of the interface are kept there. A complete manual, with annotated screens, one walkthrough for each feature and a section on how to inspect and erase what the assistant holds, will be written in the second phase once the interface has been stabilised after the defects reported in Chapter 9 have been fixed, so that the manual describes the interface that is finally delivered and not one that is still changing.

8.1 Conclusion

The user manual is deferred to the second phase for the reasons given above, so that it is written once against the stabilised interface.

turnitin Page 53 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Chapter 9 Experimental Results and Discussion

This chapter reports the experiments that were conducted in the first phase and discusses what they show about the central hypothesis, that restricting an assistant's scope to explicitly granted folders increases trust and adoption without meaningfully reducing usefulness. The controlled evaluation designed in Chapter 5, which measures task time, workload, usability and trust with participants, is scheduled for the second phase and is not reported here. What the first phase supports is a different and smaller set of questions: whether the guarantees hold in the prototype, whether the prototype performs acceptably, how it fares when used as a person would use it, and how prospective users react to the idea and to the prototype. Six experiments address these questions, and they are summarised in Table 9.1. Each is reported with its method, its results in tables and its limits.

Table 9.1: The experiments reported in this chapter.

ID	Experiment	Question answered	Evidence type
E1	Automated gate and unit tests	Do the guarantees hold under adversarial tests?	554 deterministic tests
E2	Performance measurements	Are the enforcement and retrieval costs acceptable?	Timed runs on one machine
E3	Two hands-on rounds through the interface	Does the whole system work for a user?	Scripted interface sessions
E4	Exploratory interface campaign, 27 sessions	What breaks when the system is used in a human-like and adversarial way?	Automated agents driving the real interface
E5	Observations of prospective users, 30 people	Is the idea trusted and would the product be adopted?	Reported conversations
E6	Exploratory persona simulation, 15 personas	Does the idea hold under sceptical review?	Synthetic evidence

9.1 E1: Automated Gate and Unit Tests

Method. On 9 November 2026 the development machine was used to execute the entire suite. Result. All tests passed in 42 seconds (554). The approval broker, policy engine, dispatcher and undo journal account, have 50 tests each, the speech tests have 14 tests and the remote plane has 31 and the rest of the tests are distributed among the other components as shown in Table 7.13. The claims made are that traversal, symbolic link, absolute and sibling-prefix escapes are denied and reported (TC-01); that

turnitin Page 53 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

turnitin Page 54 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

the destructive tier has no approval path and unrecognised operations are denied (TC-02); that nothing executes without a verdict and an approval (TC-03); that an approved plan reverses as an entity (TC-04); that a claim without a real source is labelled (TC-06); that voice creates no approval path (TC-09); and that a paired device cannot approve an email (TC-08). Limit. The suite demonstrates that every guarantee is true where it is used, and not that it is used everywhere. The difference is indicated in section 9.4..

9.2 E2: Performance Measurements

Method. A script measured the enforcement path and retrieval on one Windows 10 machine, using a throwaway store, folders of 100, 400 and 1,600 small text files, and the deterministic embedder. Timings for the enforcement path are medians of 300 to 2,000 repetitions. The numbers therefore describe the cost of ARTEMIS's own machinery, excluding the language model and the real embedding model, which were not part of this measurement.

Table 9.2: Enforcement and retrieval cost.

Operation	Median (ms)	95th percentile (ms)	Maximum (ms)
Broker resolves a valid reference	0.32	0.37	1.22
Broker refuses a traversal, including the audit write	1.14	2.76	27.1
Policy classifies a read	0.34	0.40	0.56
Policy refuses a delete, including the audit write	0.96	2.86	9.40
Search over 100 files	2.16	2.30	2.63
Search over 400 files	2.16	2.26	3.12
Search over 1,600 files	2.17	2.25	5.47
Verify the audit chain of 2,306 rows	10.9	–	–

Table 9.3: Time to scan and to index a folder, by size.

Files	Scan (s)	Embed and index (s)
100	0.62	0.48
400	1.96	2.26
1,600	7.98	7.64

Discussion. The boundary and the policy engine add well under a millisecond to an ordinary call, and a refusal that must also write to the hash-chained log costs about a millisecond at the median, so the

turnitin Page 54 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

turnitin Page 55 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

guarantees are not a measurable burden. Search latency stayed at about 2 milliseconds from 100 to 1,600 files, which is consistent with a single matrix product over a few thousand vectors and supports the requirement that retrieval stay fast as a folder grows, within that range. Scanning and indexing grew roughly in proportion to the number of files, at about 5 milliseconds per file for each step. Limit. Real embedding is slower than the stand-in used here, and an embedding model was not installed on the machine used for the interface campaign, so end-to-end indexing time on real hardware remains to be measured. The latency of the local language model was also not measured under controlled conditions in this phase, and it is the dominant delay a user will perceive.

9.3 E3: Hands-on Rounds Through the Interface

Method. On 8 November 2026 the whole system was driven through the real web interface in a headless browser, as a user would, and the result of each action was checked on disk. The first round used the build that had 359 unit tests passing, and a second round repeated every test after the defects had been fixed.

Table 9.4: Results of the two hands-on rounds.

Measure	Round 1	Round 2
Unit tests passing at the time	359	468
Features working from the interface	1 of 9	8 of 9 (speech not yet built)
Interface tests passing	Not applicable (first run)	36 of 36
Defects recorded	15 (one blocker, several major)	All 15 retested and fixed

Discussion. The first round found that approval buttons were hidden after a typed request, so a user could never approve anything, and that code questions, cited answers, web search, mail and drafting were unreachable from the interface even though their services passed their unit tests. The safety core nevertheless held from the start: deletion was refused, reads outside the folder were refused, and tidying and undo restored files exactly. All fifteen defects were fixed and retested on the interface, and every one of the 36 tests passed in the second round. The retest also exposed seven further problems that the fixes had introduced or revealed, among them a defect in the interface framework that froze tab clicks whenever a page-load event was used and a local model that spent its whole token budget reasoning and returned an empty answer. All seven were corrected, which is a reason the retest was run on the real interface and not only in unit tests. Limit. These rounds were scripted and run by one

10

turnitin Page 55 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

turnitin Page 56 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

tester, so they show that known paths work and say less about unanticipated use, which is what the next experiment addresses.

9.4 E4: Exploratory Interface Campaign

Method. Each of the nine areas of the prototype was tested by three automated testing agents, each driving the real web interface through a browser with a different disposition: a newcomer who follows the obvious path and makes ordinary mistakes, an impatient power user who double-clicks, refreshes and switches tabs, and an adversarial tester who supplies hostile, empty, very long and contradictory input and tries to make the system do what it promises never to do. This gave 27 sessions. Each used a private copy of the application and a private data folder, acted only through the interface, and verified outcomes on disk and in the server log. Each reported bugs and, separately, behaviour that works but that a user would not want. A consolidating step merged duplicates and compared the findings with the backlog.

Table 9.5: Session verdicts by area and stage gate status.

Area	Newcomer	Power user	Adversarial	Stage gate
Workspace foundation	Issues	Issues	Issues	Partial
Trust machinery	Issues	Issues	Fail	Partial
AI gateway	Issues	Issues	Issues	Pass
Core file features	Issues	Issues	Fail	Partial
Document and code understanding	Issues	Issues	Fail	Fail
Outward actions	Issues	Issues	Issues	Pass
Speech input and output	Issues	Fail	Issues	Pass
Remote access plane	Issues	Issues	Issues	Not tested
Conversation and everyday use	Issues	Issues	Issues	–

23.3 32 725137 1 32/ 3) 222/.2

turnitin Page 56 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

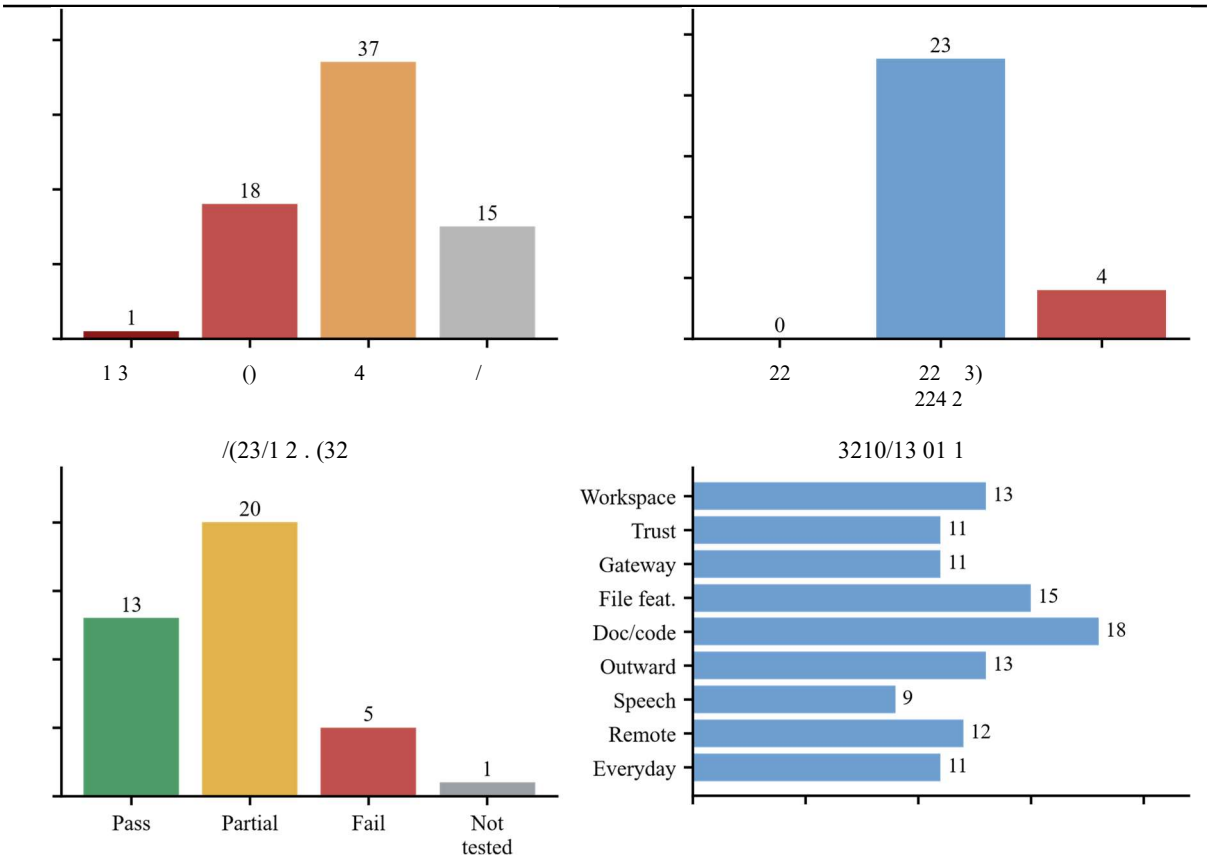


Figure 9.1: Results of the interface campaign.

Results. Figure 9.1 summarises the campaign. No session found a clean pass: none of the 27 sessions ended with the verdict pass, 23 ended with pass with issues and 4 with fail (panel b). Of the 39 backlog stories and stage gates, 13 passed, 20 passed in part, 5 failed and one could not be tested (panel c). The sessions reported 113 defects, which merged into 71 distinct defects, being 1 critical, 18 high, 37 medium and 15 low (panel a), together with 63 behaviours that work but that a user would not want. The reports were spread across all nine areas (panel d), with document and code understanding drawing the most (18) and speech input and output the fewest (9).

What held. Across many phrasings the system never deleted anything and refused any attempt to reach outside the chosen folder in plain words. Nothing left the computer, by web search or mail, without an approval card, and double clicks and stale clicks did not run a plan twice. Code questions produced only a difference and never an edit. Voice could not approve, delete or send. A phone could not approve emails or searches, saw only the folders ticked for it, and could be removed instantly. Tidy previewed every step, undo worked in ordinary use, and the rule that automation is offered only after three repetitions was exact.

What failed. The principal defects, with their severity, were the following.

- Critical. When a tidy encounters two files with the same destination name, undo will lose one of them, claiming to restore the files as they were. This is the opposite of what it promised that nothing will be lost, although there is no deletion operation, and is the first error that needs to be fixed.

- High, visibility. The history of what ARTEMIS did is on disk but not on screen, the page on which changes are reported can tell you that nothing changed when files were transferred, and a banner may remain telling that nothing had been sent since a search.

- High, boundary. The microphone might remain live with voice switched off; a plan that a phone had left waiting still executed after its folder was un-shared; a conversation might copy the contents of one folder to another; and any drive, including the system drive, could be added as a folder.

- High, trustworthy answers. Some answers provided by the system included no source, said nothing was found when it was in the files, referenced to the incorrect file, or included an email that stated it attached an attachment that was not there. Not all the gates that label unsupported claims fired.

- Medium and low. The voice panel would clutter the page, messages would be sent with the system being busy or messages sent before a reload would disappear and new files were not displayed until the user requested the system to look at the changes..

Discussion. The trend is educative. The mechanisms that are meant to support the hypothesis, consisting of the single resolution point, the tiers, the dispatcher and the undo journal, withstood adversarial use, and the majority of the severe defects are in how an interface control is wired to a gate, what a screen reports, and in cases the unit tests did not model, such as a name collision or a shared folder with a plan awaiting an allocation. This lends significance to the architectural decision to construct the enforcement first, as well as demonstrates that correctness in isolation of enforcement is not enough to provide the trust that the hypothesis predicts, as many of these defects would ruin trust in practice. Limits. The testers were not individuals but automated agents hence their reports reflect what breaks and not how an individual feels of the same. They were unable to listen to audio, could not confirm the presence of a model of cloud answer or a real email leave, did not test the side of the relay, the seven days of retaining chats or the deleting of the remote plane, and ran on a machine not embedding the model, which likely exaggerates the retrieval failures. Some of the most atrocious results are the results of a single session and these should be noted as such in the consolidated report, therefore they should be re-checked after correction. The work of the second phase is all carried over as 71 defects.

9.5 E5: Prospective User Observations.

Method. The team members had contact with 30 potential users who were students, developers, human resources employees and non-technical users. The project pitch was used to explain the idea and the participants were able to try the current prototype. They were questioned whether they could trust an assistant that works as ARTEMIS explains and they were at liberty to say what they thought about the idea and the prototype. The responses below are those reported by the team from these conversations.

Table 9.6: Stated trust, 30 prospective users.

Response	People	Share of 30
Fully agree they would trust it	10	33%
Partly agree they would trust it	16	53%
Total expressing trust	26	87%
Did not express trust	4	13%

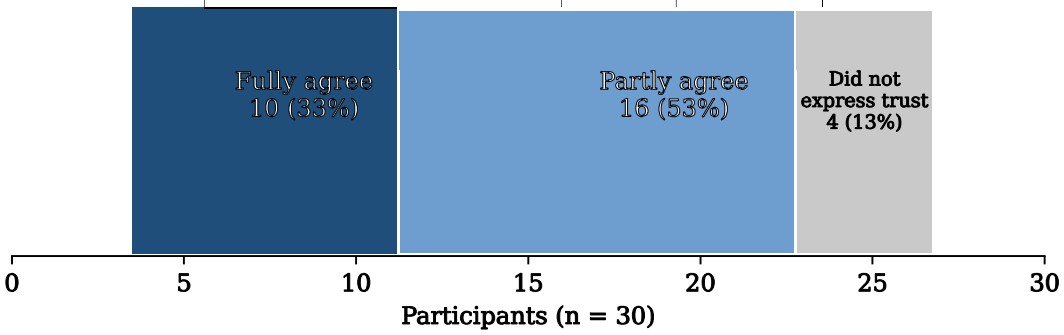


Figure 9.2: Stated trust among 30 prospective users.

Results. In general, the general concept was quite acceptable in all the four backgrounds. Twenty-six of 30 indicated trust, 10 of them entirely and the partial assent was always conditional: participants were ready to trust the assistant on the condition that it does what it claims. The participants who tested the application stated that it was a prototype of the application as it was. They dissociated this judgement and the concept and they claimed that they would be happy to embrace the end product as stipulated in the pitch.

Discussion. The status of participants which is bound to their trust is the status of the architecture being constructed to fulfil, as the trustworthiness of the assistant is supposed to be based on the boundary and on the steps in approval acting as it is promised and not on its good motives. It is also

the circumstance that the faults of the last part endanger, or, in other words, that the promise made is accompanied by the good effect, as well as by the labour that is left. The finding is similar to the underlying questionnaire where 65 percent of 374 participants found an assistant based on a workspace as more trustworthy than one based on full access, but the two questions are different, and the numbers should not be summed. Limits. The participants consist of 30 individuals who were contacted via the personal connections of the team and are not representative sample. The team collected responses in a conversation and summarised them, thus there is no data on individual responses or recordings. The answers were not counted by background and thus no figure is claimed to any of the groups. The reasons of the four participants who did not express trust were not recorded. The measures of the stated attitudes are post-pitch and short trial responses and not sustained use. They should be interpreted as pointing evidence..

9.6 E6: Exploratory Persona Simulation

Method. To test the idea under sceptical review and to expose objections that friendly conversation might not, 15 simulated personas, including students, developers, a freelancer, employees and deliberately sceptical non-adopters, were given a neutral description of the idea and a private copy of the prototype, and each reported its first impressions, what it did, its ratings and its objections in a structured report. Each worked independently and the agents did not see each other's reports. The study uses language-model-based simulated user agents to explore potential user perceptions and behavioural patterns. The participants are simulated personas and not recruited human subjects, so the results are exploratory synthetic evidence and must not be presented as empirical human-subject findings or as statistically representative of any population. They are reported here only as a stress test of the argument.

Table 9.7: Persona simulation results.

Question	Result
Would an assistant limited to chosen folders be trusted more than one with full access?	6 agree, 9 partly, 0 disagree
Would that make them more willing to use it?	1 agree, 14 partly, 0 disagree
Does the limitation cost too much usefulness?	6 no, 3 partly, 6 cannot say, 0 yes
Is wanting broad access a barrier for them?	10 agree, 2 partly, 3 disagree
Would they accept one mistake if undo is easy?	7 agree, 8 partly, 0 disagree

Ask before every change, or act and let me undo?	8 ask first, 6 mixed, 1 act then undo
Perceived relevance of the problem (1 to 5)	Mean 3.27, median 3
Perceived value of the solution (1 to 5)	Mean 2.8, median 3
Adoption	14 might try, 1 would not use, 0 would use or recommend

Discussion. The simulation is more reserved than the observations of real people, and that is its use. No persona disagreed that bounding scope would raise trust, which agrees in direction with E5, but only 1 of 15 would be clearly more willing to use it, and most attached conditions. Concerns were raised by all 15: 13 described failed, wrong or partial answers, and 12 noticed that a cloud key appeared as already saved in a clean copy, which could not be resolved as a defect or as an artefact of the shared test machine. Approval fatigue and long-term retention cannot be observed in a single session and were not. Limits. The personas are played by one underlying model, so their views are not independent, and such agents can be agreeable. The sample is small, no statistical test is used, and defects of the prototype, notably the failures of the folder box, probably coloured the personas' reactions. The results are therefore a check on the reasoning and are not evidence of adoption.

9.7 Discussion

9.7.1 Findings against the Central Hypothesis

21

Table 9.8 sets the evidence against the three parts of the central hypothesis. The classification is a description of the weight of the first-phase evidence and not a statistical test.

Table 9.8: Assessment of the central hypothesis.

Part	Evidence	Assessment
Restricting scope increases trust	Questionnaire (65 percent of 374); E5 (26 of 30, 10 fully); E6 (6 agree, 9 partly, none disagree); E4 shows the boundary mechanisms hold under hostile use	Supported in direction; conditional on exact behaviour

Increases willingness to adopt	E5: willingness to adopt the final product as pitched; E6: 14 might try, 1 clearly more willing	Partly supported; rests on stated intention
Without meaningfully reducing usefulness	E4 and E6 show the limits were not what users objected to; no task measure exists yet	Not yet tested

On this evidence the first part is supported in direction and the second is partly supported, and the third cannot be determined until usefulness is determined on the tasks. The entire hypothesis is hence promoted by the initial stage and is not confirmed yet. With the stricter interpretation of the simulation, where the only agreement that matters is the clear one, the adoption part would not support the study, and this is one of the reasons why the second phase with the controlled study should be the most weighty.

9.7.2 Results counter-pointing the Research Question.

The research question is: Does an assistant confined to given workspaces manage to provide viable multi-step task automation and enhance trust without diminishing usefulness? The prototype demonstrates that the restriction can be constructed as structure rather than policy, and can be constructed inexpensively, introducing less than a millisecond to a call. It demonstrates that, multi step plans, tidying, cited answers, mail, search, drafting, voice and remote control can be executed on the same three points of enforcement. It does not yet demonstrate that the automation is user-friendly, since that requires the task study, and the flaws of E4 indicate that reliability, and not the limitation, is what is presently limiting the experience.

9.7.3 Threats to Validity

There are four types of evidence with their respective weaknesses. The deterministic tests are unable to work out a real model. The interface campaign employed automated agents that are not able to hear or feel and one of the machines had no embedding model. The team collected the observations of 30 people, constructed the system and introduced it, which may favourably respond, and was not recorded in the individual scores. The simulation is synthetic. All these weaknesses are in different directions hence the four are reported separately and not brought together and none of them is presented as the controlled study that the second phase will offer.

9.7.4 Assessment Incomplete.

The subsequent components of the evaluation process of Chapter 5 have not been conducted and are scheduled to be implemented during the second phase: task-based measurements with the system and without it, the subjective workload measure, the usability and trust questionnaires, approval fatigue with repeated sessions, measuring the synthesis faithfulness and citation accuracy with an actual model, and the voice measure with the four-second latency target.

9.8 Conclusion

The initial stage demonstrates that the enforcement the hypothesis is based on is constructable, is run at insignificant cost and survives under adversarial use, and that potential users as questioned responded to the concept with confidence conditional on the system doing just what it says. It also demonstrates that the prototype does not yet necessarily do what it claims: the interface campaign represented 71 various defects, one of them a critical one, and the difference between a passing automated suite and a working interface was huge. The evidence validates the hypothesis direction and its aspect of trust, partially supports adoption, and measures usefulness remains to be quantified. The corrections of the defects found and the controlled study are the two main tasks of the second phase.

Chapter 10 Conclusions

10.1 Summary of the Work Done

The project aimed to demonstrate that an assistant limited to folders that the user explicitly gives, and asks before it performs and can undo whatever it did, can be trusted and used with little loss of utility to an assistant with complete access. The initial stage defined the problem and the hypothesis with a questionnaire of 374 respondents and 13 interviews, consulted 26 studies in seven themes, defined the requirements, and created an architecture which transforms the hypothesis into three enforcement points: a workspace broker which alone resolves paths, a policy engine which places every action in one tier, and a dispatcher which alone acts. It then developed a functioning prototype of some 14,250 lines in eight steps, including the workspace base, an optional remote plane, and tested it with 554 automated tests and two hand-on rounds, a 27-session interface campaign, and interviews with 30 potential users.

10.2 Findings and Results

The enforcement layer is also designed to perform as advertised and withstands adversarial usage. No deletions or accesses were made to any file during a session, no files outside the selected folder were accessible, and no files left the computer without approval, and these properties took less than a milliseconds per call. The idea was met with conditional trust by the prospective users with 26 of 30 indicating that they trusted it and 10 of them said they would embrace the end product as it was being pitched. The prototype itself is not ready for users. The interface campaign discovered 71 different bugs, one of which is a critical bug where undo following a name collision may lose a file, and demonstrated that a passing automated test can have a significantly broken interface. Responses to questions on files were occasionally not supported or incorrect, and the history on what the assistant had done was not clearly visible..

10.3 Coverage of the Scope and Objectives

Table 10.1 states, for each objective of Chapter 2, whether it has been met.

Table 10.1: Status of the project objectives.

Objective	Status	Basis and remaining work

turnitin Page 65 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Operate strictly within granted folders	Largely met	Boundary holds in all tests; any drive can still be added, a chat can carry one folder's content to another, and a waiting remote plan outlived an unshared folder.

Table 10.1: continued from previous page

Objective	Status	Basis and remaining work
Reduce time spent on resumption and routine file work	Not yet measured	Needs the task study of the second phase.
Keep the user in control through preview, approval and undo	Largely met	Gates hold; undo can lose a file on a name collision.
Provide no code path to deletion	Met	No deletion operation exists and none was reachable in any session.
Make everything held visible and deletable	Partly met	The panel works; the activity record is not fully visible.
Support voice under the same gates	Met for gates	No new gate was needed; microphone control and recognition quality need work.
Evaluate on time, workload, usability and trust	Not yet done	Planned for the second phase.

The functional scope of Chapter 2 was covered in full as a prototype: every capability listed there exists and runs through the gates, including the optional remote plane. The scope was not covered in the sense of readiness, because several capabilities are unreliable, and two of the four deliverables are not yet complete, being the user manual, which will be written once the interface has stabilised, and the evaluation report, which depends on the study of the second phase.

10.4 Challenges Faced

Four challenges shaped the first phase. The first was the gap between component tests and a working interface. The first hands-on round found that nearly every feature was unreachable although all unit tests passed, and the interface campaign later found 71 defects, which taught that the interface must be tested as a whole and that each defect needs a regression test. The second was the nondeterminism of a small local model, which sometimes gave unsupported or wrong answers, and which the gates mitigate but cannot cure. The third was the constraints of the interface framework, which required hidden controls to be triggered by scripts for some interactions and made voice behaviour, such as repeated speech, speaking the model's label, and losing the thread of a conversation, difficult to get right, and these were corrected in turn. The fourth was the size of the task for a team of three in one

turnitin Page 65 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

turnitin Page 66 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

semester, which required the highest-risk components to be built first and the evaluation to be deferred.

10.5 What Has Been Left Out and Why

The controlled evaluation, the user manual, the measurement of approval fatigue and the quality evaluation of synthesis and citations are left to the second phase, because each depends on a stable interface. Real audio, a cloud model answer, a real email and the relay's own side were not exercised in the interface campaign, because the testing agents could not hear, did not hold credentials and must not use real accounts. The 71 defects are known and recorded and have not yet been fixed.

10.6 Recommendations and Future Work

The first recommendation is to correct, before anything else, the defects that bear directly on the promises the system makes: make tidy and undo collision-safe so that a file is never overwritten and any step that cannot be restored is reported; refuse drive roots and system folders when adding a folder; stop the microphone completely when voice is off; recheck the folder grant when a waiting plan is approved from a phone; stop a chat from carrying one folder's content into another; and make the activity record and the page that reports changes truthful. The second is to make answers honest, so that an answer with no source is plainly labelled, a missing embedding model is detected and reported, and retrieval failures that say not found for facts that are present are removed. The third is to add a regression test for every defect found at the interface, and to run the interface scenarios of the campaign as part of the suite. The fourth is to carry out the controlled study of Chapter 5. Beyond the second phase, directions worth pursuing are configurable approval granularity, since the questionnaire found users evenly divided, onboarding that takes minutes and not tens of minutes, barge-in and lower latency in voice, and a check that a citation actually supports its claim and not only that it exists.

10.7 Plan for FYP-II

The second phase has about sixteen weeks. The plan in Table 10.2 orders the work so that the product is stable before it is evaluated, since a study of a prototype that is still changing would measure the changes and not the idea.

The principal risks to this plan are recruitment, which depends on the institution's networks as the risk analysis of Chapter 4 notes, and the possibility that correcting defects takes longer than allowed. The mitigation is that the order of work places stability first, so that if the study must be smaller it can be

turnitin Page 66 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

reduced in size without being run on an unstable product, and that the defect work is limited to those defects that bear directly on the project's promises.

Table 10.2: Plan of work for the second phase.

Weeks	Work	Deliverable and exit check
1 to 2	Correct the defects that bear on the promises: collision-safe tidy and undo, drive and system folder refusal, microphone control, remote grant recheck, cross-folder chat memory. Add a regression test for each.	All critical and high defects of the boundary and recovery groups closed; the suite grows by the regression tests.
3 to 4	Honest answers and visibility: label unsupported answers, detect and report a missing embedding model, repair false not-found answers, show a readable activity view and a live record of what left the computer.	Re-run of the campaign scenarios for these groups shows no repeat of the defects.
5 to 6	Onboarding and interface: guard-rails and background indexing in the folder box, voice panel layout, no silently lost messages, visibility of new files. Voice latency and barge-in.	Medium defects of the interface group closed; time for a new user to reach a first answer measured.
7	Re-run of the full interface campaign, including the remote plane's relay side, retention of chats and removal of the remote plane.	Updated defect register; stage gates that were not tested are now tested.
8	User manual (Chapter 8).	Complete manual written from the stabilised interface.
9 to 10	Design the controlled study: tasks, instruments for workload, usability and trust, consent and ethics approval, recruitment through the institution. Pilot with a small group.	Study protocol approved by the supervisor; pilot completed and the instruments revised.
11 to 13	Conduct the study: task completion with and without the system, repeated sessions for approval fatigue, measurement of synthesis faithfulness and citation accuracy, voice latency against the foursecond target.	Data collected for the planned number of participants.
14	Analyse the results and classify the central hypothesis and each supporting claim against the criteria fixed beforehand.	Analysis tables and figures.

15	Complete the report: full Chapter 9, final Chapter 10, updated manual.	Final report draft to the supervisor.
16	Final fixes, demonstration preparation and defence.	Demonstration build and final submission.

10.8 Conclusion of the Report

The initial step has yielded a prototype where the key concept, that the area of an assistant can be restricted by design and its actions can be reviewed and undone at low cost, has been put into practice and is tested under hostile conditions at negligible cost, and potential users who were consulted have responded with confidence that relied on the system doing precisely what it claims to. The same phase revealed that the prototype is yet to do so everywhere, and the defects are documented. It is only left to ensure that the product is reliable and measure, among the participants and in tasks, whether the trust earned is worth an acceptable cost in utility. The second phase is that..

Bibliography

[1] LangChain, "LangGraph: Building stateful, multi-actor applications with LLMs." Available: https://langchain-ai.github.io/langgraph/ [Accessed September 06, 2026], 2024.

[2] "Ollama: Get up and running with large language models locally." Available: https://ollama. com/ [Accessed September 06, 2026], 2024.

[3] SQLite Consortium, "SQLite: Self-contained, serverless, zero-configuration, transactional SQL database engine." Available: https://www.sqlite.org/ [Accessed September 06, 2026], 2024.

[4] United Nations, "Transforming our world: The 2030 agenda for sustainable development." Available: https://sdgs.un.org/2030agenda [Accessed September 06, 2026], 2015.

[5] S. T. Iqbal and E. Horvitz, "Disruption and recovery of computing tasks: Field study, analysis, and directions," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '07)*, pp. 677–686, ACM, 2007. DOI: 10.1145/1240624.1240730.

[6] D. Hu and S. W. Lee, "ScreenTrack: Using a visual history of a computer screen to retrieve documents and web pages," in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*, pp. 1–13, ACM, 2020. DOI: 10.1145/3313831.3376753.

[7] C. Paduraru, P.-L. Bouruc, and A. Stefanescu, "A trace-based assurance framework for agentic AI orchestration: Contracts, testing, and governance," in *Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2026)*, SciTePress, 2026.

[8] J. Kim, X. Liu, Z. Wang, S. Qiu, B. Li, W. Guo, and D. Song, "The attack and defense landscape of agentic AI: A comprehensive survey," in *Proceedings of the 35th USENIX Security Symposium (USENIX Security '26)*, 2026.

[9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

turnitin Page 70 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

[10] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[11] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, pp. 1–22, ACM, 2023. DOI: 10.1145/3586183.3606763.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*, 2023.

[13] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[14] A. M. Deshmukh and R. Chalmeta, "User experience and usability of voice user interfaces: A systematic literature review," *Information*, vol. 15, no. 9, p. 579, 2024. DOI: 10.3390/info15090579.

[15] F. L. I. Dutsinma, D. Pal, S. Funilkul, and J. H. Chan, "A systematic review of voice assistant usability: An ISO 9241–11 approach," *SN Computer Science*, vol. 3, no. 4, p. 267, 2022. DOI: 10.1007/s42979-022-01172-3.

[16] Y. Ma, X. Zhang, X. Zhang, N. Bart!omiejczyk, S. Je, A. Holzer, M. Fjeld, and A. M. Butz, "Beyond words: Measuring user experience through speech analysis in voice user interfaces," in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, ACM, 2026. DOI: 10.1145/3772318.3791747.

[17] X. Zhang, Y. Ma, K. Li, Y. Zhou, M. Fjeld, and M. Obaid, "Exploring the design of human speech indicators to enhance waiting experience in voice user interface," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*, ACM, 2025. DOI: 10.1145/3706598.3713090.

[18] J. Wang, X. Zhang, Y. Ma, and M. Fjeld, "Mitigating response delays in free-form conversations with LLM-powered intelligent virtual agents," in *Proceedings of the 7th ACM Conference on Conversational User Interfaces (CUI '25)*, ACM, 2025. DOI: 10.1145/3719160.3736636.

[19] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li,

turnitin Page 70 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proceedings of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '25)*, 2025.

[20] E. A. Gonzalez, R. Rothkopf, S. Lerner, and N. Polikarpova, "HiLDe: Intentional code gener-´ ation via human-in-the-loop decoding," in *Proceedings of the 2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '25)*, 2025.

[21] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "When to show a suggestion? integrating human feedback in AI-assisted programming," in *Proceedings of the 38th AAAI Conference on ArtificialIntelligence(AAAI-24)*, vol. 38, pp. 10137–10144, 2024. DOI: 10.1609/aaai.v38i9.28878.

[22] S. Chidambaram, L. E. Li, M. Bai, X. Li, K. Lin, X. Zhou, and A. C. Williams, "Socratic human feedback (SoHF): Expert steering strategies for LLM code generation," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 15491–15502, 2024. Available: https://aclanthology.org/2024.findings-emnlp.908/.

[23] H. Jin and H. Chen, "Are LLMs reliable code reviewers? systematic overcorrection in requirement conformance judgement," *Automated Software Engineering*, vol. 33, p. 90, 2026. DOI: 10.1007/s10515-026-00638-5.

[24] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pp. 1906–1919, 2020. DOI: 10.18653/v1/2020.acl-main.173.

[25] W. Krysci´ nski, B. McCann, C. Xiong, and R. Socher, "Evaluating the factual consistency of ab-´ stractive text summarization," in *Proceedingsofthe2020ConferenceonEmpiricalMethodsinNatural Language Processing (EMNLP 2020)*, pp. 9332–9346, 2020. DOI: 10.18653/v1/2020.emnlpmain.750.

[26] A. Pagnoni, V. Balachandran, and Y. Tsvetkov, "Understanding factuality in abstractive summarization with FRANK: A benchmark for factuality metrics," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2021)*, pp. 4812–4829, 2021. DOI: 10.18653/v1/2021.naaclmain.383.

[27] P. Laban, W. Krysci´ nski, D. Agarwal, A. R. Fabbri, C. Xiong, S. Joty, and C.-S. Wu, "SummEdits:´ Measuring LLM ability at factual reasoning through the lens of summarization," in *Proceedings of*

the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023), pp. 9662–9676, 2023. DOI: 10.18653/v1/2023.emnlp-main.600.

[28] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, "Measuring attribution in natural language generation models," *Computational Linguistics*, vol. 49, no. 4, pp. 777–840, 2023. DOI: 10.1162/coli a 00486.

[29] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pp. 6465–6488, 2023. DOI: 10.18653/v1/2023.emnlp-main.398.

[30] N. F. Liu, T. Zhang, and P. Liang, "Evaluating verifiability in generative search engines," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 7001–7025, 2023. DOI: 10.18653/v1/2023.findings-emnlp.467.

turnitin Page 73 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Chapter A Research Instruments and Response Distributions

This appendix records the instruments used in the foundational research and the complete response distributions summarised in Chapters 2 and 3. It is included so that the findings cited throughout the report can be examined at the level of the underlying data.

A.1 Research Method

The foundational research comprised two instruments. A questionnaire of seventeen items drew 374 valid responses, covering the respondent's context, the problem, design and trust preferences, and feature demand, with attitudinal items using a five-point scale. An interview guide coded thirteen in-person conversations on five axes: the last occasion on which the participant lost their place in multi-file work, the workaround they already employ, their reaction to the concept, the effect a trust-breaking mistake would have, and whether they would continue using the system.

Recruitment was through the institution's student and practitioner networks, and the questionnaire ran from 22 to 27 August 2026. Four limitations apply and are stated because they bound every conclusion drawn from this data. The sample is self-selected and drawn from a single institution. Students predominate, so the professional signal rests on a smaller subsample. The system was described in text rather than demonstrated, so predicted use and predicted churn are stated intentions rather than observed behaviour. Some free-text responses contain near-identical phrasings, which suggests that certain answers were presented as suggested text, so theme counts should be read as indicative.

A.2 Respondent Profile

Table A.1 gives the composition of the sample and the frequency with which respondents perform multifile work.

Table A.1: Respondent role and work frequency.

Response	Count	Percent
Role		
Student	302	80.7
Developer or software engineer	47	12.6
Freelance	12	3.2

turnitin Page 73 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

Employed	7	1.9
Other	6	1.6

Table A.1: continued from previous page

Response	Count	Percent
Frequency of multi-file project work		
Daily	147	39.3
A few times a week	159	42.5
A few times a month	68	18.2
Prior use of an AI assistant		
Yes, regularly	172	46.0
Yes, occasionally	115	30.7
Tried once or twice	48	12.8
Never	39	10.4

A.3 Headline Findings

The nine findings established by the research, referenced throughout Chapters 2 and 4, are as follows.

- F1. The reorientation problem is real. 44 percent of respondents struggle to recall what they were doing on returning to a project after a break, and only 7 percent never do
- F2. File disorder outpaces manual cleanup for 59 percent of respondents
- F3. The problem costs real time at moderate scale, with 42 percent losing more than an hour a week and 14 percent more than three hours
- F4. The work is unautomated because of distrust of automation at 36 percent and lack of knowhow at 28 percent, rather than because the tasks do not exist
- F5. The bounded-workspace hypothesis holds. 65 percent rate a workspace-only assistant more trustworthy than a full-access one, with a mean of 3.8, and 40 percent prefer local-first operation
- F6. Users are forgiving of a single mistake. 65 percent would continue using the system after one wrong suggestion and only 10 percent would abandon it
- F7. The principal retention risks are habit fit at 27 percent and an untrusted mistake at 24 percent; approval fatigue is real but smaller at 10 percent

turnitin Page 75 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

- F8. Feature demand is concentrated on session resume, notes synthesis and folder tidy, with speech last by a wide margin
- F9. The interviews confirm the questionnaire and add effortless onboarding as a named condition

  for continued use

A.4 Segment Analysis

Table A.2 presents selected measures by respondent segment. Role does not predict the interaction preference, since students and developers give near-identical workspace-trust ratings and identical approvalfirst shares. Experience moves it modestly: regular users of assistants prefer approval-first less often than light or non-users, which is consistent with heavier users being more comfortable delegating. The effect is small and neither group approaches unanimity, so the practical conclusion is unchanged, which is that the interaction model must be a setting rather than a fixed decision.

Table A.2: Selected measures by respondent segment.

Segment	n	Workspace trust (mean)	Prefer approval-first	Prefer local-first
All respondents	374	3.8	51%	40%
Students	302	3.8	51%	41%
Developers and engineers	47	3.7	51%	36%
Regular assistant users	172	3.9	44%	40%
Light or non-users	87	3.8	59%	41%

A.5 Interview Findings

Figure A.1 presents the coded interview responses. Ten of the thirteen participants reacted positively to the concept, with the remainder split between conditional, mixed and one negative response. Eight of thirteen said a single mistake would not stop them, several qualifying this with caution, while two said a trust-breaking mistake would cause them to stop. Six said outright that they would continue using the system and four said they would do so conditionally, with effortless onboarding named as the most common condition, followed by privacy and by the system reaching sufficient maturity.

turnitin Page 75 of 78 - Integrity Submission Submission ID trn:oid:::3971:158451361

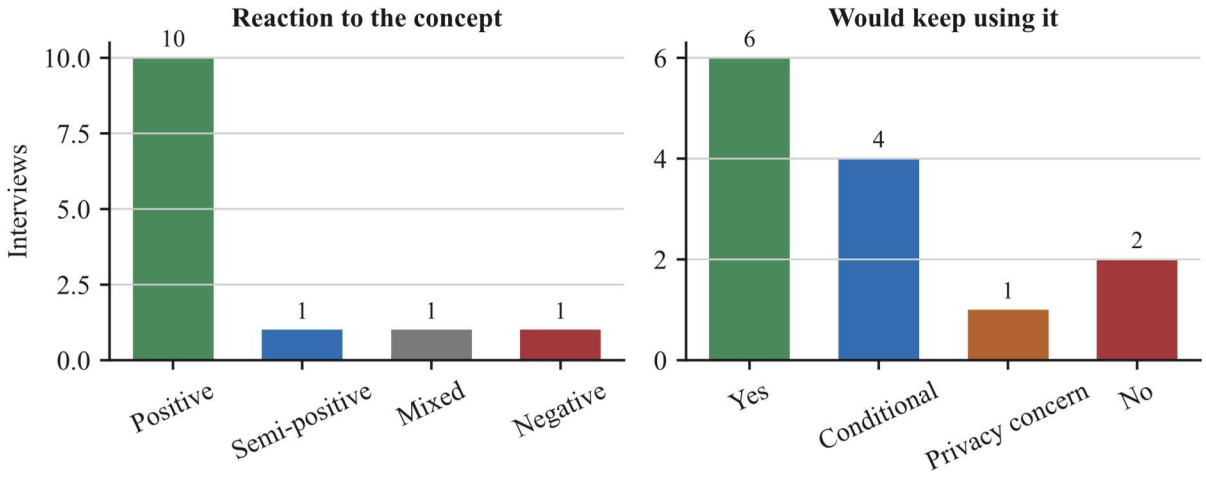


Figure A.1: Coded interview responses (13 interviews).

The workarounds participants already employ were consistent: informal notes written by hand or with an assistant to record where they left off, repeated manual restructuring because every project has a different structure, and simply tolerating the friction because automating it appears to cost more than it saves.

A.6 Additional Response Distributions

Figures A.2, A.3, A.4, A.5, A.6, A.7 and A.8 present the remaining distributions underlying the findings cited in Chapters 2 and 4.

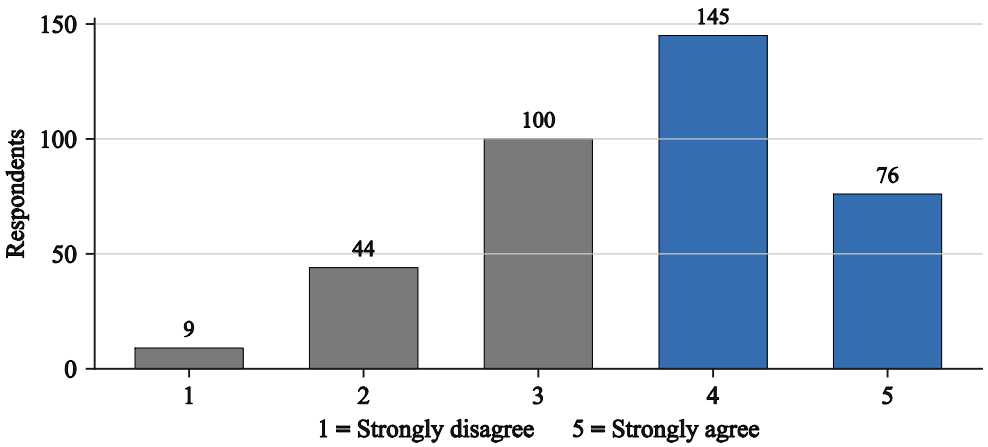


Figure A.2: Agreement that files become disorganised.

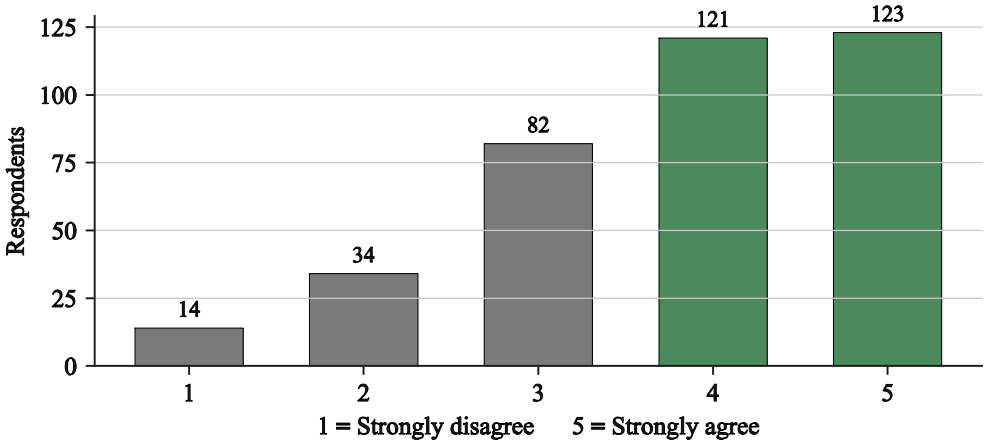


Figure A.3: Agreement on workspace-only trust.

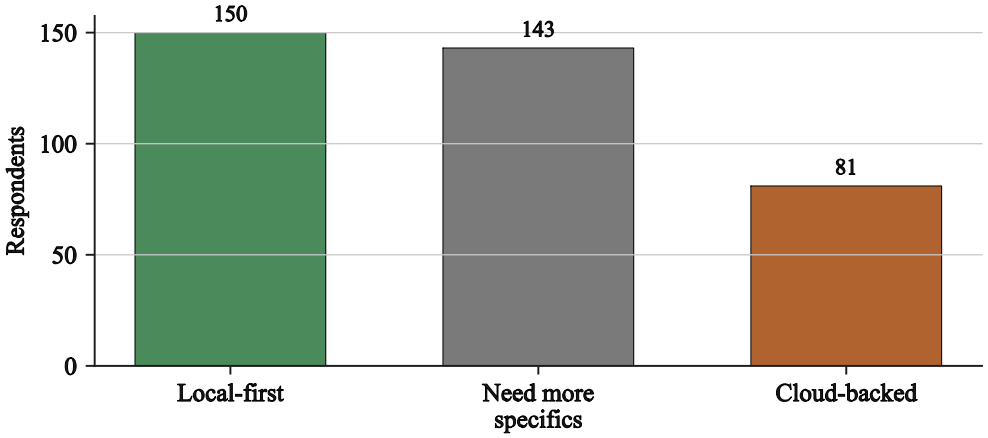


Figure A.4: Local versus cloud preference.

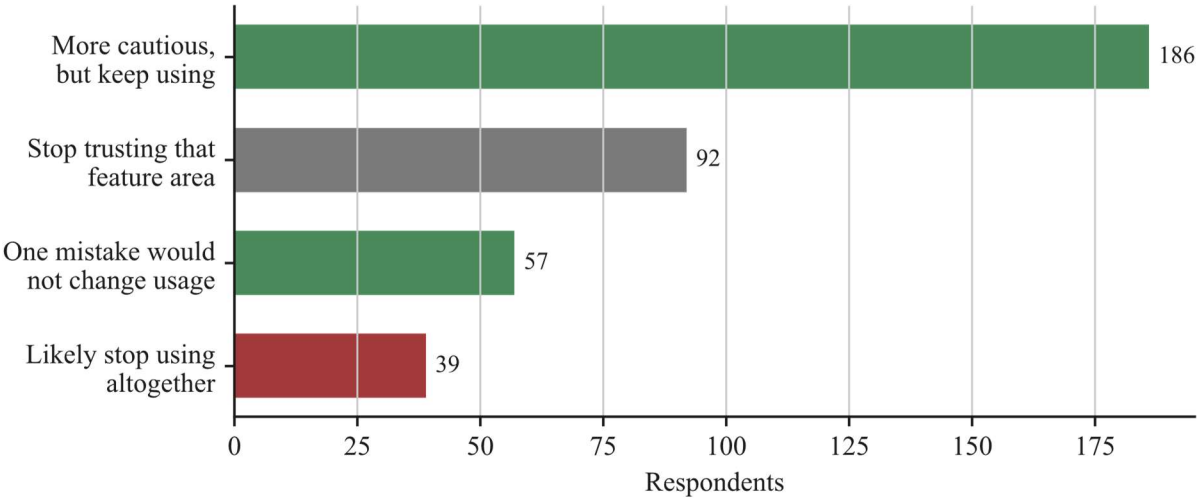


Figure A.5: Response to a single wrong or unwanted suggestion.

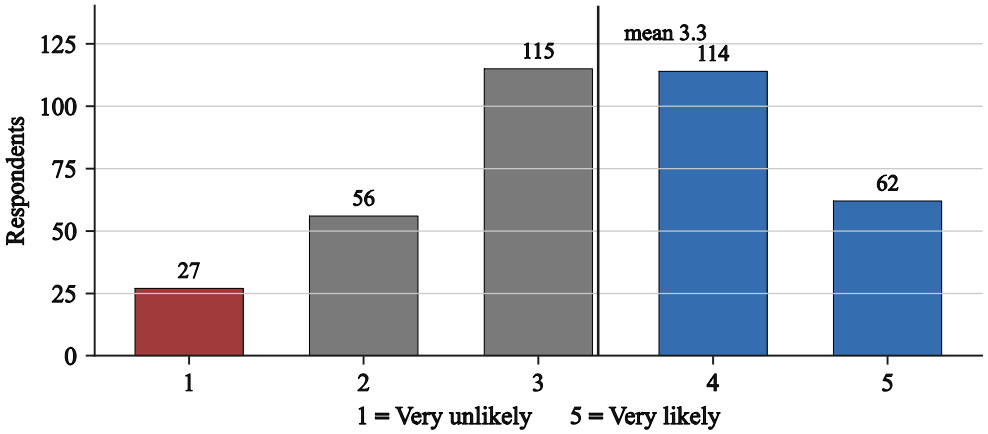


Figure A.6: Likelihood of continued use after the novelty.

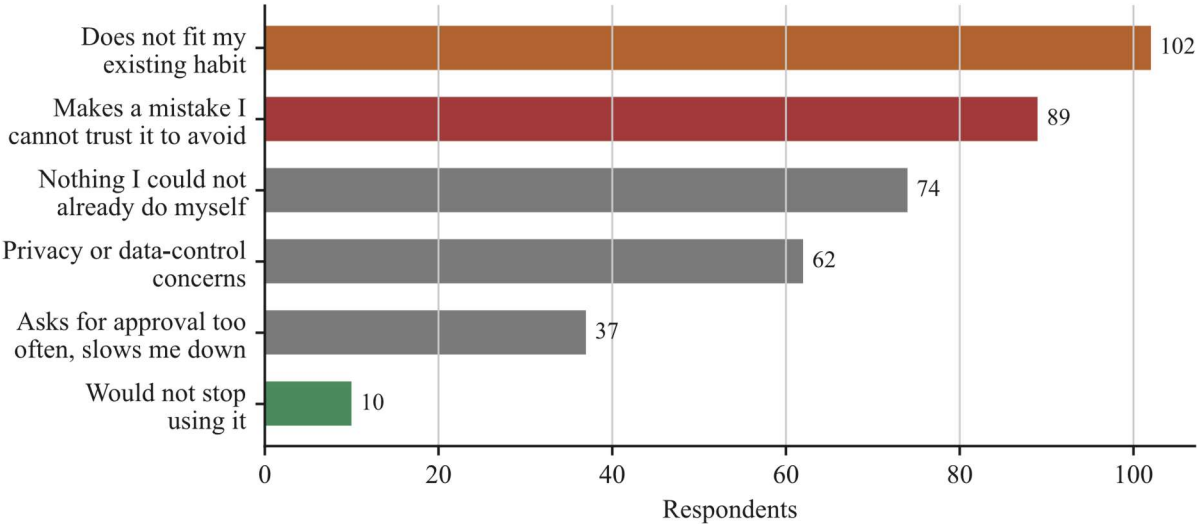


Figure A.7: Most likely reason for stopping after a week.

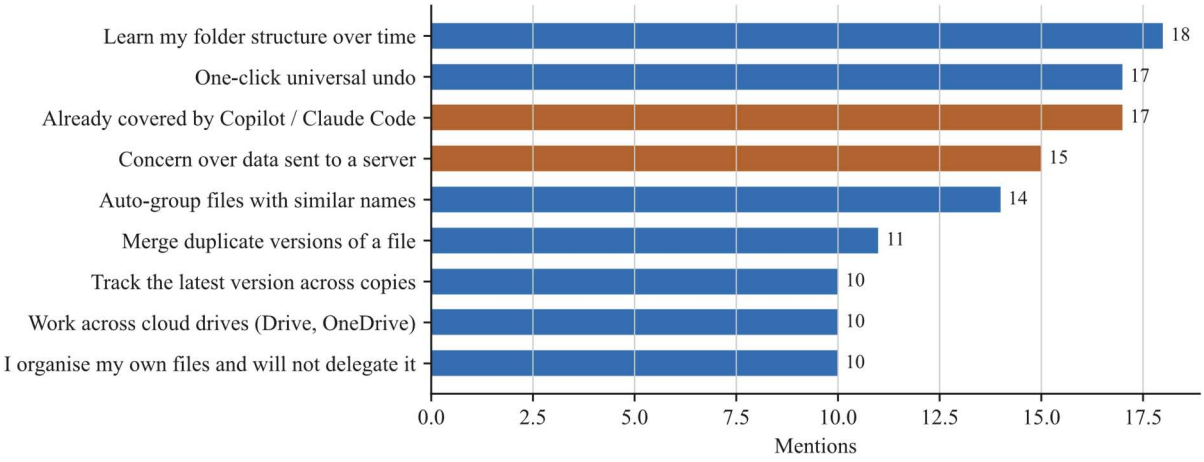


Figure A.8: Recurring themes in free-text responses.